

———— // ZPWR-MIDI-FX // ————

zpwr-midi-fx - Block Reference

v0.3.0

MODULE REFERENCE

note-stream MIDI effect modules

2026-07-29



MENKE
TECHNOLOGIES



MTPUBLISHING



MTAUDIO

Contents

Catalog at a glance	2
Architecture	3
Performance	4
Reading this catalog (badge key)	5
MIDI (zpw-midi-fx) (563 blocks)	6
Control (5)	6
Dynamics (162)	6
Expression (3)	23
Harmony (51)	23
Modulation (60)	28
Probability (180)	34
Routing (29)	53
Sequencing (63)	56
Timing (2)	62
Tuning (3)	62
Voicing (5)	63
Index (563 blocks, alphabetical)	64

Catalog at a glance

563 blocks shipped by zpwr-midi-fx across 11 categories, including 0 component-level analog-circuit models and 248 control / modulation (CV-badge) sources. This is the per-plugin subset of the shared zpwr-patch-core catalog.

Registry MIDI 563
Output jack Audio 0 MIDI 563 CV 0

By category (block count, largest first):

Probability.....	180	Control.....	5
Dynamics.....	162	Voicing.....	5
Sequencing.....	63	Expression.....	3
Modulation.....	60	Tuning.....	3
Harmony.....	51	Timing.....	2
Routing.....	29		

Architecture

Every block in this catalog runs through one shared patch-graph engine. The full signal and control path - from the host buffer, through the layered engine and the per-voice patch graph, onto the shared bus, and back out - is below. Signal flows top to bottom; control and modulation enter from the right; the engine's CPU optimizations are called out where they apply.

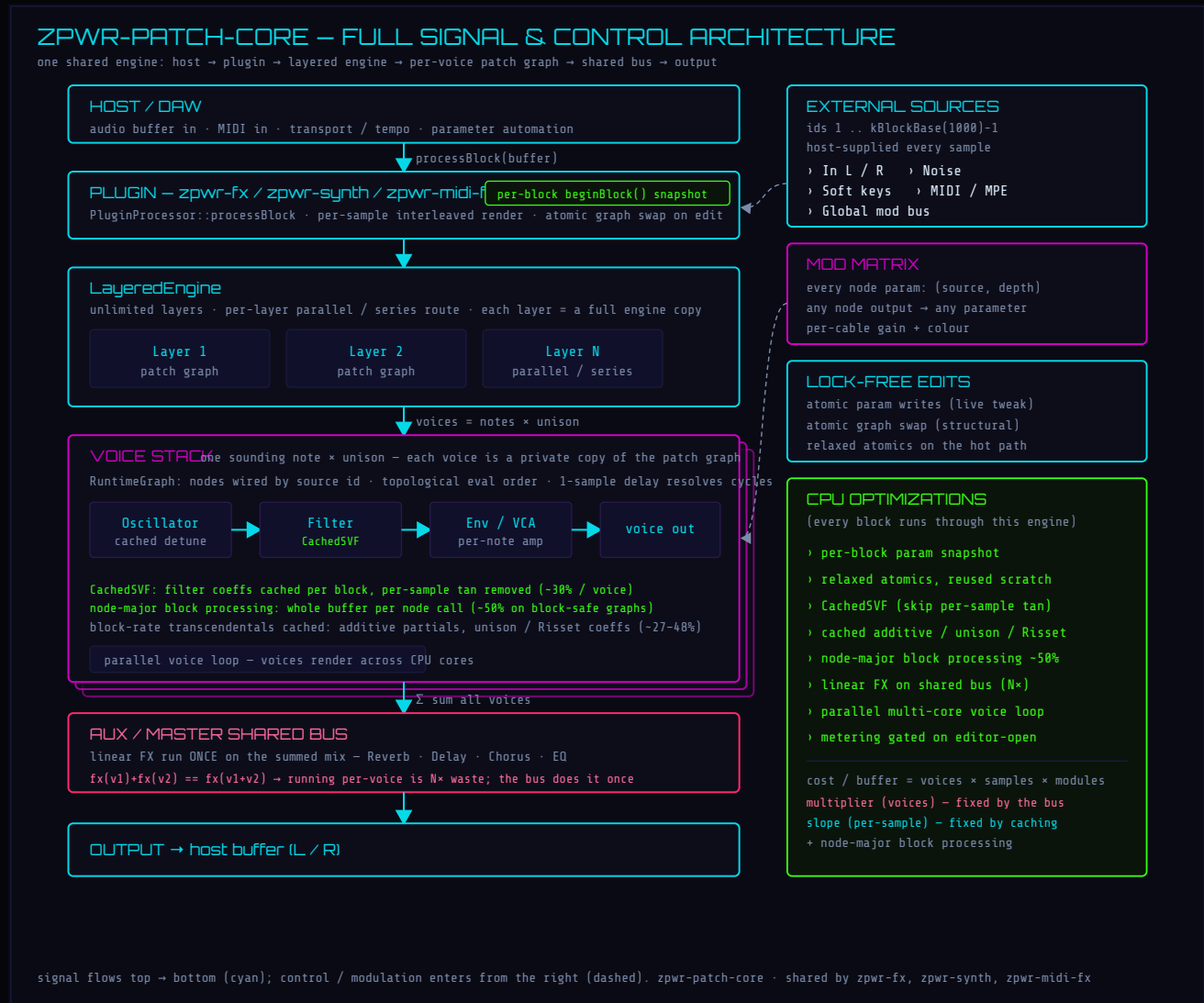


Figure 1: Full signal & control architecture of zpwr-patch-core

Performance

Every block in this catalog runs through one shared patch-graph engine, so every optimization in that engine speeds up every patch. The full list of CPU work shipped to date is below, grouped by what it attacks. The cost of one audio buffer is voices x samples x modules: the multiplier (voices x per-voice work) is what pins a chord; the slope (per-sample dispatch and coefficient recompute) is the constant overhead. The phases below attack each in turn. Measured figures are arm64, -O3, from interleaved before/after benchmark runs.

1. **Engine hot path** - shaves the per-sample slope; together ~18% off per-sample DSP:

- > **Relaxed atomic ordering** on every hot-path load (params, sync-divisor, gain, bypass, bus-send, modulation), dropping the arm64 acquire barrier on the inner loop.
- > **Reused compute-context scratch** - a single member buffer reused each sample instead of constructing one per sample.
- > **Tempo-sync scan skipped** for nodes that have no syncable parameter.
- > **Per-block parameter snapshot** - on the host's `beginBlock` call, unmodulated nodes do zero atomic parameter loads per sample; a safe per-sample fallback runs when a host never calls it.

2. **Coefficient & transcendental caching** - block-rate values that used to be recomputed every sample are now computed once per block and reused, recomputed only when an input parameter changes (all bit-identical):

- > **CachedSVF** caches the state-variable (TPT) filter coefficients, skipping a per-sample `tan`; one subclass fixes every SVF-based block with no call-site edits. ChoirPad 3-voice: 106 to 76 ns/sample (~30%).
- > **Additive partial-amplitude cache** - a dense additive voice was running up to 70 `pow` per sample (14 partials x 5 internal voices); recomputed only on a Partial / Rolloff / Odd-Even change. 1140 to 600 ns/sample (~48%).
- > **Unison detune-ratio cache** - a `pow` plus a `sqrt` per voice per sample, cached once; benefits every unison oscillator. Osc 5-voice: 246 to 180 ns/sample (~27%).
- > **Per-voice detune-ratio cache** for Screech, Hoover, Reese, Wt, SynthFM and SynthSync (the per-voice `pow(2, spread x detune)` ratio).
- > **Risset decay coefficients** - the seven-partial `exp` decay loop cached.
- > **Additator amplitude-table cache** - up to 32 `pow` per sample on block-rate Partial / Tilt / Balance, cached like the additive table.

3. **Voice multiplier** - the real fix for chord cost; turns N copies of an effect back into one:

- > **Linear FX kept on the shared bus**. Reverb and delay are linear (`fx(v1)+fx(v2) == fx(v1+v2)`), so running them per voice is N-times waste for a bit-identical result. They live on the shared aux / master bus; only synthesis and per-note timbre stay in the voice patch.
- > **Every factory preset audited clean** - no linear FX sits in any voice layer.
- > **Regression gate** - the preset validator rejects any reverb placed in a voice layer, so the multiplier cannot creep back in.

4. **Host & parallelism** - removes wasted per-voice work in the synth host:- **Metering gated on editor-open** - the per-sample `O(nodes)` metering scan is skipped entirely while the editor is closed. - **Parallel voice loop** - voices render across CPU cores through a persistent worker pool, bit-identical to serial, opt-in.

5. **Block (node-major) processing** - each node processes a whole buffer in one call instead of one sample at a time, amortizing the per-sample cable-gather, context-build and dispatch overhead (~50% on block-safe graphs; a per-sample-capture fallback covers graphs with buses or modulation):

- > **11 FX blocks converted** to whole-buffer processing - the medium FX. Each mirrors its per-sample body over the buffer with block-rate coefficients hoisted out of the loop, and is verified bit-identical to the per-sample path. Converted so far: Bitcrush, Chorus, Comb, Drive, EQ, Filter, Flanger, Gate, Lofi, Tremolo, Waveshaper.

Reading this catalog (badge key)

An icon glyph precedes each block name; then its badges, then **in:** its input jacks and **out:** its output-jack type, then the description:

- > **[CAT]** **category** - **AMP** Amp, **ANA** Analog, **CTL** Control, **DLY** Delay, **DST** Distortion, **DYN** Dynamics, **ENV** Envelope, **FLT** Filter, **FX** Effect, **HRM** Harmony, **MOD** Modulation, **OSC** Oscillator, **PIT** Pitch, **PRB** Probability, **REV** Reverb, **RTE** Routing, **SEQ** Sequencing, **SPC** Spectral, **STR** Stereo, **TUN** Tuning, **UTL** Utility, **VOI** Voicing.
- > **[CV]** a **control / modulation source** (outputs CV, not audio).
- > **[CIRCUIT]** a **component-level analog-circuit model** (per-sample nodal / Newton solve).
- > **[PHY]** a **physical-model instrument** (waveguide / modal / friction synthesis of strings, membranes and resonant bodies).
- > **[FX]** **[SYNTH]** **[MIDI]** which **plugin(s)** ship the block - **zpwr-fx**, **zpwr-synth** (gets the whole audio pack) and/or **zpwr-midi-fx**.
- > **in:** the block's input jacks, and **out:** its output-jack signal type - **out:** **Audio**, **out:** **MIDI** (note stream) or **out:** **CV** (control / modulation).
- > **icon:** the glyph before each block name shows its kind at a glance:

	Oscillator		Filter		Delay
	Reverb		LFO / mod		Envelope
	Dynamics		Stereo		Pitch
	Spectral		Harmony		Sequencer
	Routing		Voicing		Effect
	Control		Probability		Tuning
	Utility		Guitar amp		Pedal / stompbox
	Speaker cabinet		Analog circuit		Other

MIDI (zpw-midi-fx) (563 blocks)

Control (5)

🕒 **ATProc** [CTL][CV][MIDI] in: Audio out: MIDI

Scales aftertouch, optionally converting it to a CC.

🕒 **BendProc** [CTL][CV][MIDI] in: Audio out: MIDI

Scales and/or inverts incoming pitch-bend.

🕒 **CC2Note** [CTL][CV][MIDI] in: Audio out: MIDI

A watched CC crossing a threshold triggers a note.

🕒 **CCMap** [CTL][CV][MIDI] in: Audio out: MIDI

Remaps a CC number and scales/offsets its value.

🕒 **Note2CC** [CTL][CV][MIDI] in: Audio out: MIDI

Emits a CC from each note (pitch or velocity).

Dynamics (162)

⌋ **AbundanceVelocity** [DYN][MIDI] in: Audio out: MIDI

Abundance velocity: accents each note by the signed abundance $\sigma(n)-2n$, so deficient counts read soft, perfect counts sit at centre and abundant counts read loud; Depth blends with the played velocity.

⌋ **AbundancyVelocity** [DYN][MIDI] in: Audio out: MIDI

Abundancy velocity: drives velocity from the abundancy index $\sigma(n)/n$ of the count - near 1 for primes, climbing past 2 for perfect and abundant numbers - so divisor-rich counts hit hardest; Depth blends with the played velocity.

⌋ **Accent** [DYN][MIDI] in: Audio out: MIDI

Boosts velocity on every Nth note.

⌋ **AccentEveryN** [DYN][MIDI] in: Audio out: MIDI

Accent every N: boosts the velocity of every Nth note and slightly ducks the rest, stamping a steady metric downbeat onto an even stream - the simplest way to imply a time signature in the dynamics.

└ **AccentOnHigh** [DYN] [MIDI] in: Audio out: MIDI

Accent on high: boosts the velocity of note-ons at or above a pitch threshold, brightening the top of the keyboard.

└ **AccentOnLow** [DYN] [MIDI] in: Audio out: MIDI

Accent on low: boosts the velocity of note-ons at or below a pitch threshold, reinforcing the bass register.

└ **AccentPattern** [DYN] [MIDI] in: Audio out: MIDI

Accent pattern: boosts velocity on the on-steps of a 16-step accent bitmask and trims it elsewhere, stamping a repeating dynamic groove onto an even stream of notes.

└ **AliquotStepsVelocity** [DYN] [MIDI] in: Audio out: MIDI

Aliquot-steps velocity: accents each note by how many steps the count's aliquot sequence (repeatedly summing proper divisors) takes before terminating or settling, an unpredictable number-theory accent; Depth blends with the played velocity.

└ **AliquotVelocity** [DYN] [MIDI] in: Audio out: MIDI

Aliquot velocity: drives velocity from the aliquot sum (the sum of a number's proper divisors) of the note count, the quantity behind perfect, abundant and deficient numbers; Depth blends with the played velocity.

└ **AlternatingSumVelocity** [DYN] [MIDI] in: Audio out: MIDI

Alternating-sum velocity: drives velocity from the alternating digit sum of the count (the signed digit total behind the divisible-by-11 test), an oscillating accent contour; Depth blends with the played velocity.

└ **BackbeatAccent** [DYN] [MIDI] in: Audio out: MIDI

Backbeat accent: boosts the velocity of notes on beats 2 and 4 (the backbeat) and slightly softens the others, stamping the rock/pop snare-accent feel onto a flat stream without dropping notes.

└ **BeattyVelocity** [DYN] [MIDI] in: Audio out: MIDI

Beatty velocity: drives velocity from the Beatty sequence $\text{floor}(n \cdot \sqrt{2})$ (an irrational-rotation staircase that, with its complement, partitions the integers); Depth blends with the played velocity.

└ **BernoulliSignVelocity** [DYN] [MIDI] in: Audio out: MIDI

Bernoulli-sign velocity: alternates two velocity levels by the sign of the Bernoulli numbers (the alternating-sign even-index sequence from number theory and calculus), with a neutral level on the zero-valued odd indices.

└ **BinaryRunVelocity** [DYN] [MIDI] in: Audio out: MIDI

Binary-run velocity: accents each note by the longest run of identical bits in its binary count, spiking on counts like 7, 15, 31 and 56; Depth blends with the played velocity.

└ **BinaryWeightVelocity** [DYN] [MIDI] in: Audio out: MIDI

Binary-weight velocity: drives velocity from the number of 1-bits (Hamming weight) in the note count, a self-similar accent pattern that pulses with the binary structure of the ordinal; Depth blends it with the played velocity.

└ **CalkinWilfVelocity** [DYN] [MIDI] in: Audio out: MIDI

Calkin-Wilf velocity: drives velocity from the Calkin-Wilf rational at the count ($\text{fusc}(n)/\text{fusc}(n+1)$), the breadth-first enumeration that lists every positive fraction exactly once; Depth blends with the played velocity.

└ **CatalanVelocity** [DYN] [MIDI] in: Audio out: MIDI

Catalan velocity: steps velocity through the Catalan numbers (1,1,2,5,14,42,...) taken modulo a range, the combinatorial sequence counting balanced brackets and binary trees; Depth blends with the played velocity.

└ **CollatzOddStepsVelocity** [DYN] [MIDI] in: Audio out: MIDI

Collatz-odd-steps velocity: accents each note by how many odd ($3n+1$) rises its count takes on the way down the Collatz trajectory, an erratic hailstone-flavoured accent; Depth blends with the played velocity.

└ **CollatzPeakVelocity** [DYN] [MIDI] in: Audio out: MIDI

Collatz-peak velocity: accents each note by the log-height of the highest value its count reaches in the Collatz ($3n+1$) hailstone trajectory, so counts that soar before falling hit hardest; Depth blends with the played velocity.

└ **CollatzVelocity** [DYN] [MIDI] in: Audio out: MIDI

Collatz velocity: maps each note's velocity to the number of $3n+1$ steps its count takes to reach 1, an erratic deterministic accent sequence from the unsolved conjecture; Depth blends against the played velocity.

└ **CompositeRunVelocity** [DYN] [MIDI] in: Audio out: MIDI

Composite-run velocity: accents each note by how many composite counts have passed since the last prime, ramping up through prime gaps and snapping back to zero on each prime; Depth blends with the played velocity.

└ **CototientVelocity** [DYN] [MIDI] in: Audio out: MIDI

Cototient velocity: accents each note by the cototient n minus Euler's totient (the count of integers up to n that share a factor with it), a divisor-flavoured accent complementary to the totient; Depth blends with the played velocity.

└ **CrescendoLoop** [DYN] [MIDI] in: Audio out: MIDI

Crescendo loop: velocity climbs from Min to Max across Length notes then snaps back and repeats, an automatic sawtooth dynamic swell driven by the note count rather than the clock.

└ **CrescendoRamp** [DYN] [MIDI] in: Audio out: MIDI

Crescendo ramp: ramps note-on velocity up linearly across a phrase of N notes then resets, an automatic swell from soft to loud; Depth blends with the played velocity.

└ **DigitalRootVelocity** [DYN] [MIDI] in: Audio out: MIDI

Digital-root velocity: accents each note by the digital root (the repeated digit-sum, 1..9) of its count, a perfectly periodic nine-step accent staircase from elementary number theory; Depth blends with the played velocity.

└ **DigitEntropyVelocity** [DYN] [MIDI] in: Audio out: MIDI

Digit-entropy velocity: drives velocity from how many distinct decimal digits the count uses, so repdigits read soft and varied counts read loud; Depth blends with the played velocity.

└ **DigitMaxVelocity** [DYN] [MIDI] in: Audio out: MIDI

Digit-max velocity: drives velocity from the largest decimal digit of the note count, a coarse stepping accent that climbs and resets with the leading digits; Depth blends with the played velocity.

└ **DigitMinVelocity** [DYN] [MIDI] in: Audio out: MIDI

Digit-min velocity: drives velocity from the smallest decimal digit of the note count, the mirror of digit-max; Depth blends with the played velocity.

└ **DigitProductVelocity** [DYN] [MIDI] in: Audio out: MIDI

Digit-product velocity: accents each note by the product of its count's decimal digits, so counts with big digits hit hard and those with small digits stay soft; Depth blends with the played velocity.

└ **DigitReverseVelocity** [DYN] [MIDI] in: Audio out: MIDI

Digit-reverse velocity: drives velocity from the count read backwards (123 becomes 321), an erratic accent that scrambles the steady climb of the counter into a jumpy pattern; Depth blends with the played velocity.

└ **DigitRotateVelocity** [DYN] [MIDI] in: Audio out: MIDI

Digit-rotate velocity: rotates the count's decimal digits one place (the last digit jumps to the front) and reads velocity from the result, scrambling the steady counter into a jumpy pattern; Depth blends with the played velocity.

└ **DigitSpreadVelocity** [DYN] [MIDI] in: Audio out: MIDI

Digit-spread velocity: drives velocity from the spread between the largest and smallest decimal digit of the note count, so repdigit counts read soft and mixed-digit counts read loud; Depth blends with the played velocity.

└ **DigitSumVelocity** [DYN] [MIDI] in: Audio out: MIDI

Digit-sum velocity: derives velocity from the decimal digit-sum of the note count, a self-similar sawtooth-of-sawtooths accent that resets every power of ten; Depth blends it with the incoming velocity.

└ **DiminuendoRamp** [DYN] [MIDI] in: Audio out: MIDI

Diminuendo ramp: ramps note-on velocity down linearly across a phrase of N notes then resets, an automatic fade from loud to soft; Depth blends with the played velocity.

└ **DistinctPrimeVelocity** [DYN] [MIDI] in: Audio out: MIDI

Distinct-prime velocity: accents each note by little-omega, the number of distinct prime factors of its count (primes softest, highly-composite counts loudest); Depth blends with the played velocity.

└ **DivisorCountVelocity** [DYN] [MIDI] in: Audio out: MIDI

Divisor-count velocity: makes notes louder on counts with many divisors, so highly-composite ordinals (12, 24, 36...) land as accents and primes stay quiet; Depth blends against the played velocity.

└ **DivisorProductVelocity** [DYN] [MIDI] in: Audio out: MIDI

Divisor-product velocity: drives velocity from the product of the note count's divisors (taken modulo a range), a number-theoretic accent that swings high for highly-composite counts; Depth blends with the played velocity.

└ **DivisorSumVelocity** [DYN] [MIDI] in: Audio out: MIDI

Divisor-sum velocity: drives velocity from $\sigma(n)$, the sum of all divisors of the note count, so primes read low and highly-composite counts read loud; Depth blends with the played velocity.

└ **EuclideanAccentVelocity** [DYN] [MIDI] in: Audio out: MIDI

Euclidean accent: boosts the velocity of notes landing on a Euclidean (Bjorklund) rhythm and softens the rest, stamping an evenly-distributed accent groove onto a steady stream of notes without dropping any.

└ **EuclideanVelocity** [DYN] [MIDI] in: Audio out: MIDI

Euclidean velocity: accents the notes that fall on the onsets of a Euclidean rhythm of Pulses-in-Steps and softens the rest, stamping a maximally-even world-rhythm groove onto the dynamics.

└ **ExpDecayVelocity** [DYN] [MIDI] in: Audio out: MIDI

Exp-decay velocity: velocity starts loud and decays exponentially across a phrase of N notes then resets, an automatic decrescendo per phrase; Depth blends with the played velocity.

└ **FactorialBaseVelocity** [DYN] [MIDI] in: Audio out: MIDI

Factorial-base velocity: accents each note by the digit-sum of its count written in the factorial number system (where place values are $1!, 2!, 3!, \dots$), an exotic mixed-radix accent contour; Depth blends with the played velocity.

└ **FactorialModVelocity** [DYN] [MIDI] in: Audio out: MIDI

Factorial-mod velocity: drives velocity from the count's factorial taken modulo 127, which collapses to zero past a small index (Wilson's theorem territory), giving a sharp early flurry then silence-velocity; Depth blends with the played velocity.

└ **FibVelocity** [DYN] [MIDI] in: Audio out: MIDI

Fibonacci velocity: drives note velocity from the Fibonacci sequence taken modulo 128, climbing then wrapping in the golden cadence; Depth blends the pattern against the incoming velocity.

└ **GcdVelocity** [DYN] [MIDI] in: Audio out: MIDI

GCD velocity: sets velocity from the greatest common divisor of the note count and a modulus, so counts sharing big factors with the modulus hit harder - a quietly-periodic, factor-driven accent; Depth blends with the played velocity.

└ **GhostNoteEveryN** [DYN] [MIDI] in: Audio out: MIDI

Ghost-note every N: softens every Nth note-on down to a quiet ghost-note level, dropping a recurring accent into the background for a syncopated groove.

└ **GhostNoteInject** [DYN] [MIDI] in: Audio out: MIDI

Ghost-note inject: randomly demotes a fraction of notes to a soft 'ghost' level, scattering the quiet in-between hits that give drum and bass parts their human, shuffling feel.

└ **GoldbachCountVelocity** [DYN] [MIDI] in: Audio out: MIDI

Goldbach-count velocity: accents each even count by the number of distinct ways it splits into a sum of two primes (its Goldbach partitions), a rising and jagged accent; Depth blends with the played velocity.

└ **GoldenAngleVelocity** [DYN] [MIDI] in: Audio out: MIDI

Golden-angle velocity: rotates each note count by the 137.5-degree golden angle (the phyllotaxis spiral that arranges sunflower seeds) and reads velocity from the resulting angle, an evenly-scattering accent; Depth blends with the played velocity.

└ **GoldenSectionAccent** [DYN] [MIDI] in: Audio out: MIDI

Golden-section accent: places a strong accent at the start and at the golden-ratio point (~0.618) of a repeating cycle, an aesthetically-balanced asymmetric stress pattern instead of an even backbeat.

└ **GoldenVel** [DYN] [MIDI] in: Audio out: MIDI

Golden velocity: replaces each note's velocity with a value from the golden-ratio low-discrepancy sequence, which fills the dynamic range far more evenly than random, so accents never clump; Depth blends it against the played velocity.

└ **GrayCodeVelocity** [DYN] [MIDI] in: Audio out: MIDI

Gray-code velocity: drives velocity from the reflected-binary Gray code of the note count (successive values differ by one bit), giving a smoothly-rotating, single-step-change accent pattern; Depth blends with the played velocity.

└ **HammingDistanceVelocity** [DYN] [MIDI] in: Audio out: MIDI

Hamming-distance velocity: drives velocity from how many bits flip between successive note counts (the Hamming distance of consecutive integers), a small spiky accent that jumps at binary carries; Depth blends with the played velocity.

└ **HappyVelocity** [DYN] [MIDI] in: Audio out: MIDI

Happy velocity: accents each note by how its count behaves under the happy-number process (sum of squared digits) - loud and bright if it reaches 1, darker if it falls into the sad cycle; Depth blends with the played velocity.

└ **HarmonicNumberVelocity** [DYN] [MIDI] in: Audio out: MIDI

Harmonic-number velocity: drives velocity from the harmonic number $H(n)=1+1/2+...+1/n$, a smooth slowly-saturating logarithmic climb across the note count; Depth blends with the played velocity.

└ **HarmonicVelocity** [DYN] [MIDI] in: Audio out: MIDI

Harmonic velocity: shapes velocity along a $1/(k+1)$ harmonic-decay curve over a repeating length-note cycle, so each phrase opens strong and tapers - an automatic accent-and-decay groove.

└ **Humanize** [DYN] [MIDI] in: Audio out: MIDI

Adds random timing and velocity jitter for a human feel.

└ **HumanizeVelocity** [DYN] [MIDI] in: Audio out: MIDI

Humanize velocity: adds a small random variation to each note-on velocity so repeated hits no longer read as identical, a subtle natural-feel dither.

└ **IntervalVelocity** [DYN] [MIDI] in: Audio out: MIDI

Interval velocity: sets each note's velocity from the size of its leap from the previous note, so wide jumps hit hard and stepwise motion stays gentle - dynamics that follow the melodic contour; Depth blends with the played velocity.

└ **JacobiVelocity** [DYN] [MIDI] in: Audio out: MIDI

Jacobi-symbol velocity: picks one of three velocity levels from the Jacobi symbol of the count over an odd Modulus (+1, -1 or 0), a quadratic-residue signature from number theory; the modulus sets the pattern's period.

└ **JacobsthalVelocity** [DYN] [MIDI] in: Audio out: MIDI

Jacobsthal velocity: drives velocity from the Jacobsthal numbers ($J(n)=J(n-1)+2J(n-2)$: 1,1,3,5,11,21,...), a Fibonacci cousin tied to powers of two; Depth blends with the played velocity.

└ **KaprekarVelocity** [DYN] [MIDI] in: Audio out: MIDI

Kaprekar velocity: drives velocity from how many digit-sort-and-subtract steps the count takes to reach Kaprekar's constant 6174, an erratic 0-7 accent from a famous digit routine; Depth blends with the played velocity.

└ **KolakoskiVelocity** [DYN] [MIDI] in: Audio out: MIDI

Kolakoski velocity: alternates two velocity levels following the self-describing Kolakoski sequence (whose run-lengths are the sequence itself), giving a hypnotically self-similar yet non-repeating accent pattern.

└ **LeonardoVelocity** [DYN] [MIDI] in: Audio out: MIDI

Leonardo velocity: drives velocity from the Leonardo numbers ($L(n)=L(n-1)+L(n-2)+1$: 1,1,3,5,9,15,25,...), the smoothsort sequence kin to Fibonacci; Depth blends with the played velocity.

└ **LogVelocity** [DYN] [MIDI] in: Audio out: MIDI

Log velocity: drives velocity from the base-2 logarithm of the note count, a gentle ever-slowing ramp that doubles its reach each octave of counts; Depth blends with the played velocity.

└ **LookSayVelocity** [DYN] [MIDI] in: Audio out: MIDI

Look-and-say velocity: drives velocity from the growing length of the look-and-say sequence (1, 11, 21, 1211, ...) at the note count, a self-describing sequence that lengthens by Conway's constant; Depth blends with the played velocity.

└ **LucasVelocity** [DYN] [MIDI] in: Audio out: MIDI

Lucas velocity: steps velocity through the Lucas numbers (the Fibonacci companion 2,1,3,4,7,11,...) taken modulo a range, a golden-cadence accent contour; Depth blends with the played velocity.

└ **MarkovVelocity** [DYN] [MIDI] in: Audio out: MIDI

Markov velocity: walks the velocity through a near-neighbour Markov chain of levels, so dynamics drift coherently (mostly small changes, occasional leaps) rather than jumping randomly - a stochastic but musical accent contour.

└ **MelodicAccent** [DYN] [MIDI] in: Audio out: MIDI

Melodic accent: accents notes that step up from the previous one and softens those that step down, so the dynamics follow the rise and fall of the melodic contour - phrasing that breathes with the line.

└ **MobiusVelocity** [DYN] [MIDI] in: Audio out: MIDI

Moebius velocity: picks one of three velocity levels from the Moebius function μ of the count - high for squarefree-even-factor counts, low for squarefree-odd, neutral for counts with a squared factor - a number-theoretic three-state accent.

└ **ModInverseVelocity** [DYN] [MIDI] in: Audio out: MIDI

Mod-inverse velocity: drives velocity from the modular inverse of the note count under a chosen modulus (zero when the count shares a factor with the modulus), a number-theoretic scramble; Depth blends with the played velocity.

└ **ModRangeVelocity** [DYN] [MIDI] in: Audio out: MIDI

Mod-range velocity: ramps velocity in a repeating sawtooth across a chosen period of notes, a steady rising-then-resetting accent staircase; Depth blends with the played velocity.

└ **MoebiusVelocity** [DYN] [MIDI] in: Audio out: MIDI

Moebius velocity: accents notes by the Moebius function of the count - loud on squarefree numbers with an even number of prime factors, soft on odd, and mid-level on square-divisible ones; Depth blends with the played velocity.

└ **NarayanaVelocity** [DYN] [MIDI] in: Audio out: MIDI

Narayana velocity: drives velocity from Narayana's-cows sequence ($a(n)=a(n-1)+a(n-3)$: 1,2,3,4,6,9,13,19,...), whose ratio tends to the supergolden ratio; Depth blends with the played velocity.

└ **NibbleSwapVelocity** [DYN] [MIDI] in: Audio out: MIDI

Nibble-swap velocity: scrambles velocity by swapping the high and low 4-bit nibbles of the note count, a byte-twiddling accent that jumps in a fixed but jagged pattern; Depth blends with the played velocity.

└ **OmegaVelocity** [DYN] [MIDI] in: Audio out: MIDI

Omega velocity: sets velocity from the number of prime factors (with multiplicity) of the count, so smooth primes play soft and factor-heavy numbers play hard; Depth blends it with the incoming velocity.

└ **PadovanVelocity** [DYN] [MIDI] in: Audio out: MIDI

Padovan velocity: steps velocity through the Padovan sequence ($P(n)=P(n-2)+P(n-3)$, the plastic-number recurrence) taken modulo a range, a slow, gently-rolling accent contour; Depth blends with the incoming velocity.

└ **PartitionVelocity** [DYN] [MIDI] in: Audio out: MIDI

Partition velocity: drives velocity from $p(n)$, the number of ways to write the count as a sum of positive integers, a fast-growing combinatorial accent; Depth blends with the played velocity.

└ **PellVelocity** [DYN] [MIDI] in: Audio out: MIDI

Pell velocity: drives velocity from the Pell numbers ($P(n)=2P(n-1)+P(n-2)$: 1,2,5,12,29,70,...), the silver-ratio cousin of Fibonacci; Depth blends with the played velocity.

└ **PerrinVelocity** [DYN] [MIDI] in: Audio out: MIDI

Perrin velocity: drives velocity from the Perrin sequence ($P(n)=P(n-2)+P(n-3)$: 3,2,3,2,5,5,7,10,...), famous for its primality-test conjecture; Depth blends with the played velocity.

└ **PersistenceVelocity** [DYN] [MIDI] in: Audio out: MIDI

Persistence velocity: accents each note by the multiplicative-persistence depth of its count (how many digit-product steps reach a single digit), a sparse, spiky accent rooted in a famous open digit problem; Depth blends with the played velocity.

└ **PitchClassVelocity** [DYN] [MIDI] in: Audio out: MIDI

Pitch-class velocity: accents every note of a chosen pitch class and softens the rest, so a tonic or any scale degree rings out across the whole keyboard - a harmonic spotlight on one note name.

└ **PopcountVelocity** [DYN] [MIDI] in: Audio out: MIDI

Popcount velocity: drives velocity from the number of 1-bits in the note count (its binary Hamming weight), a jagged bit-pattern accent; Depth blends with the played velocity.

└ **PowerOfTwoFactorVelocity** [DYN] [MIDI] in: Audio out: MIDI

Power-of-two-factor velocity: drives velocity from the largest power of two that divides the count, so odd counts read soft and highly-even counts read loud (a binary-divisibility accent); Depth blends with the played velocity.

└ **PrimeCountVelocity** [DYN] [MIDI] in: Audio out: MIDI

Prime-count velocity: drives velocity from the prime-counting function $\pi(\text{count})$ - how many primes are at or below the note count - taken modulo a span, a slowly-climbing-then-wrapping accent rooted in the distribution of primes.

└ **PrimeFactorVelocity** [DYN] [MIDI] in: Audio out: MIDI

Prime-factor velocity: sets velocity from the largest prime factor of the count (log-scaled), so prime counts ring loud and smooth, small-factored counts stay soft; Depth blends with the played velocity.

└ **PrimeGapVelocity** [DYN] [MIDI] in: Audio out: MIDI

Prime-gap velocity: drives velocity from the size of the prime gap bracketing the note count (the distance between the primes just below and just above it), so notes near large prime deserts hit harder; Depth blends with the played velocity.

└ **PrimeIndexAccent** [DYN] [MIDI] in: Audio out: MIDI

Prime-index accent: accents the prime-numbered notes in the stream (the 2nd, 3rd, 5th, 7th, 11th, ... to pass) and softens the rest, an irregular number-theoretic accent pattern.

└ **PrimeIndexVelocity** [DYN] [MIDI] in: Audio out: MIDI

Prime-index velocity: drives velocity from the n -th prime number (the prime sitting at the running count's index), an irregularly-climbing accent straight from the prime sequence; Depth blends with the played velocity.

└ **PrimePiVelocity** [DYN] [MIDI] in: Audio out: MIDI

Prime-pi velocity: drives velocity from the prime-counting function $\pi(\text{count})$ - how many primes are at or below the count - a slowly-climbing staircase accent from analytic number theory; Depth blends with the played velocity.

└ **PrimeVelocity** [DYN] [MIDI] in: Audio out: MIDI

Prime velocity: drives velocity from the gaps between successive prime numbers, an irregular yet deterministic accent sequence that never settles into an obvious loop; Depth blends it with the incoming velocity.

└ **PrimorialModVelocity** [DYN] [MIDI] in: Audio out: MIDI

Primorial-mod velocity: drives velocity from the primorial (product of the first k primes) taken modulo 127, a fast-scrambling accent that jumps as each new prime multiplies in; Depth blends with the played velocity.

└ **RadicalVelocity** [DYN] [MIDI] in: Audio out: MIDI

Radical velocity: accents each note by the radical of its count (the product of its distinct prime factors), so squarefree counts read high and prime-power counts read low; Depth blends with the played velocity.

└ **Ramp** [DYN] [MIDI] in: Audio out: MIDI

Velocity ramp over a run of notes, ascending or descending.

└ **RandomHoldVelocity** [DYN] [MIDI] in: Audio out: MIDI

Random-hold velocity: picks a random velocity and holds it for Hold notes before re-rolling, a sample-and-hold dynamic that gives blocky terraced changes rather than per-note jumps; Depth blends with the played velocity.

└ **RegisterAccent** [DYN] [MIDI] in: Audio out: MIDI

Register accent: boosts the velocity of notes inside a Low..High pitch window and softens those outside, spotlighting one register's dynamics without dropping any notes.

└ **RepeatDecay** [DYN] [MIDI] in: Audio out: MIDI

Repeat decay: each immediate repetition of the same note plays quieter by a decay factor, taming machine-gun retriggers into a natural fade and resetting the moment the pitch changes.

└ **ReverseBitsVelocity** [DYN] [MIDI] in: Audio out: MIDI

Bit-reverse velocity: drives velocity from the 8-bit bit-reversal of the note count, scrambling the steady counter into the scattered order used by FFT bit-reversal; Depth blends with the played velocity.

└ **ReverseSubtractVelocity** [DYN] [MIDI] in: Audio out: MIDI

Reverse-subtract velocity: drives velocity from the absolute difference between the count and its digit-reversal (the 196-algorithm / Kaprekar step), zero on palindromes and large on lopsided counts; Depth blends with the played velocity.

└ **RoughnessVelocity** [DYN] [MIDI] in: Audio out: MIDI

Roughness velocity: drives velocity from the smallest prime factor of the note count, the complement of smoothness, so even counts read low and prime-like counts read high; Depth blends with the played velocity.

└ **RudinShapiroAccent** [DYN] [MIDI] in: Audio out: MIDI

Rudin-Shapiro accent: pushes each note's velocity up or down by the +/-1 Rudin-Shapiro sequence of its count, an aperiodic accent pattern with a famously flat (white-like) spectrum - structured stress that never repeats predictably.

└ **RulerVelocity** [DYN] [MIDI] in: Audio out: MIDI

Ruler velocity: accents notes by the ruler sequence (the power of two dividing the count: 0,1,0,2,0,1,0,3,...), so every other note is a light tick and the downbeats of each binary level land harder - a self-similar metric accent.

└ **RunningSumVelocity** [DYN] [MIDI] in: Audio out: MIDI

Running-sum velocity: accumulates a small per-note increment into a velocity that drifts upward and wraps, a slowly-cycling ramp untied to pitch; Depth blends with the played velocity.

└ **ScaleDegreeAccent** [DYN] [MIDI] in: Audio out: MIDI

Scale-degree accent: accents notes by their function in the key - tonic loudest, then fifth and third, with weak and chromatic degrees softened - carving a dynamic shape that follows the harmony.

└ **ScaleHighlight** [DYN] [MIDI] in: Audio out: MIDI

Scale highlight: accents notes that belong to the chosen Root/Scale and softens the chromatic outsiders, spotlighting the key without removing any notes - a scale-aware dynamic emphasis.

└ **SigmaVelocity** [DYN] [MIDI] in: Audio out: MIDI

Sigma velocity: drives velocity from $\text{sigma}(\text{count})$, the sum of all divisors of the note count, so highly-divisible counts hit harder - an accent contour straight out of number theory; Depth blends with the played velocity.

└ **SineWaveVelocity** [DYN] [MIDI] in: Audio out: MIDI

Sine-wave velocity: shapes velocity as a smooth sine across a chosen period of notes, a gentle rise-and-fall dynamic swell repeating every Period notes; Depth blends with the played velocity.

└ **SmoothnessVelocity** [DYN] [MIDI] in: Audio out: MIDI

Smoothness velocity: drives velocity from the largest prime factor of the note count, so smooth (small-factor) counts read soft and prime counts read loud; Depth blends with the played velocity.

└ **SquareDistanceVelocity** [DYN] [MIDI] in: Audio out: MIDI

Square-distance velocity: accents notes by how close the count sits to a perfect square - loud right on the squares, fading in the gaps - a pulsing accent that widens as the squares spread apart; Depth blends with the played velocity.

└ **SquareRootVelocity** [DYN] [MIDI] in: Audio out: MIDI

Square-root velocity: drives velocity from the integer square root of the note count, a staircase whose steps grow ever wider; Depth blends with the played velocity.

└ **SquareWaveVelocity** [DYN] [MIDI] in: Audio out: MIDI

Square-wave velocity: alternates every note between two fixed velocity levels, a hard 2-step on/off accent that drives a mechanical pumping groove.

└ **StaircaseVelocity** [DYN] [MIDI] in: Audio out: MIDI

Staircase velocity: velocity climbs in quantized steps up a staircase across a chosen number of steps then drops back, a terraced crescendo accent; Depth blends with the played velocity.

└ **SternBrocotVelocity**_[DYN]_[MIDI] in: Audio out: MIDI

Stern-Brocot velocity: shapes velocity from Stern's diatomic (fusc) sequence, the fractal numerators of the Stern-Brocot tree of rationals, giving a self-similar sawtooth-of-sawtooths accent; Depth blends with the incoming velocity.

└ **SternDiatomicVelocity**_[DYN]_[MIDI] in: Audio out: MIDI

Stern-diatomic velocity: accents each note by Stern's diatomic (fusc) sequence 1,1,2,1,3,2,3,1,4,... whose consecutive ratios enumerate every rational once (the Stern-Brocot tree); Depth blends with the played velocity.

└ **SwingVelocity**_[DYN]_[MIDI] in: Audio out: MIDI

Swing velocity: accents on-beat notes and softens off-beat ones, applying the dynamic side of a swing/shuffle feel (the loud-soft alternation) independent of timing - instant groove from even input.

└ **TernaryDigitVelocity**_[DYN]_[MIDI] in: Audio out: MIDI

Ternary-digit velocity: drives velocity from the base-3 digit sum of the note count, a self-similar accent that climbs and resets on ternary carries; Depth blends with the played velocity.

└ **ThueMorseTernaryVel**_[DYN]_[MIDI] in: Audio out: MIDI

Ternary Thue-Morse velocity: picks one of three velocity levels from the base-3 Thue-Morse sequence (digit-sum mod 3) of the count, a self-similar three-level accent that avoids short repeats; Depth blends with the played velocity.

└ **ThueMorseVelocity**_[DYN]_[MIDI] in: Audio out: MIDI

Thue-Morse velocity: alternates two velocity levels by the cube-free Thue-Morse parity of the note count, accenting in a self-similar, never-quite-repeating pattern instead of a simple every-other.

└ **TotientRatioVelocity**_[DYN]_[MIDI] in: Audio out: MIDI

Totient-ratio velocity: drives velocity from Euler's totient ratio $\phi(n)/n$, so primes read near-full and highly-composite counts read soft, an arithmetic density accent; Depth blends with the played velocity.

└ **TotientStepsVelocity**_[DYN]_[MIDI] in: Audio out: MIDI

Totient-depth velocity: accents each note by how many times Euler's totient must be applied to its count to reach 1 (the iterated-totient depth, roughly log-scaled); Depth blends with the played velocity.

└ **TotientVelocity**_[DYN]_[MIDI] in: Audio out: MIDI

Totient velocity: drives velocity from Euler's totient ratio $\phi(n)/n$ of the count, dipping on numbers rich in small prime factors and peaking on primes - a number-theoretic accent contour; Depth blends with the played velocity.

└ **TrailingOnesVelocity**_[DYN]_[MIDI] in: Audio out: MIDI

Trailing-ones velocity: accents each note by how many 1-bits trail at the bottom of its binary count (0,1,0,2,0,1,0,3,...), a carry-driven spiky pattern; Depth blends with the played velocity.

└ **TriangularModVelocity** [DYN][MIDI] in: Audio out: MIDI

Triangular-mod velocity: shapes velocity as a triangle wave rising and falling across a chosen period of notes, a smooth swell-and-fade accent contour; Depth blends with the played velocity.

└ **TribonacciVelocity** [DYN][MIDI] in: Audio out: MIDI

Tribonacci velocity: drives velocity from the Tribonacci numbers where each term sums the previous three (0,1,1,2,4,7,13,24,44,...); Depth blends with the played velocity.

└ **TurnaroundAccent** [DYN][MIDI] in: Audio out: MIDI

Turnaround accent: boosts the velocity of notes that are local melodic turning points - the peaks and valleys where the line changes direction - emphasising the contour's corners.

└ **UnitaryDivisorVelocity** [DYN][MIDI] in: Audio out: MIDI

Unitary-divisor velocity: accents each note by the sum of its count's unitary divisors (divisors that share no factor with their cofactor), a coprime-divisor variant of the sigma accent; Depth blends with the played velocity.

└ **VelByPitch** [DYN][MIDI] in: Audio out: MIDI

Velocity by pitch: sets each note's velocity from its keyboard position (a Slope per semitone around middle C plus a Base), so high notes can ring louder or softer - keyboard-position dynamics.

└ **VelClip** [DYN][MIDI] in: Audio out: MIDI

Clamps velocity between Min and Max.

└ **VelCompress** [DYN][MIDI] in: Audio out: MIDI

Velocity compressor: velocities above Threshold are pulled toward it by Ratio (1 = off, 8 = hard), narrowing the dynamic range smoothly instead of hard-clamping like VelClip.

└ **VelCurve** [DYN][MIDI] in: Audio out: MIDI

Reshapes velocity through a power curve.

└ **VelDrift** [DYN][MIDI] in: Audio out: MIDI

Velocity drift: each note's velocity takes one step of a bounded random walk, so dynamics wander smoothly and correlatedly over time instead of jumping independently like Humanize - the slow swell and ebb of a tiring player.

└ **VelGate** [DYN][MIDI] in: Audio out: MIDI

Velocity gate: drops any note whose velocity falls outside the Min..Max window, so only soft (or only hard) hits pass - a dynamics-dependent filter for splitting ghosted from accented playing.

└ **VelGateHysteresis** [DYN] [MIDI] in: Audio out: MIDI

Velocity Schmitt gate: opens once a note exceeds the High velocity and stays open until one drops below Low, so soft passages between accents either all pass or all mute - hysteretic dynamic gating that avoids chattering at the threshold.

└ **VelInvert** [DYN] [MIDI] in: Audio out: MIDI

Flips the velocity scale around its midpoint (soft <-> loud) by Amount, so your accents become ghost notes and the ghosts become accents - a velocity mirror, blendable to taste.

└ **VelKey** [DYN] [MIDI] in: Audio out: MIDI

Keyboard velocity tracking: scales each note's velocity by how far its pitch sits from a Center note, Tilt setting how strongly and which way - higher notes louder (or softer), the dynamic key-tracking of real instruments.

└ **Vellength** [DYN] [MIDI] in: Audio out: MIDI

Note duration scales with velocity: hard notes ring for Max ms, soft notes for Min (set Max < Min to make accents staccato) - the played articulation where dynamics shape note length.

└ **Velocity** [DYN] [MIDI] in: Audio out: MIDI

Scales and offsets note velocity, with a modulation amount.

└ **VelocityAlternate** [DYN] [MIDI] in: Audio out: MIDI

Velocity alternate: alternates note-on velocities between two fixed levels (loud, soft, loud, soft), an automatic backbeat-style dynamic groove.

└ **VelocityBoost** [DYN] [MIDI] in: Audio out: MIDI

Velocity boost: adds a fixed amount (positive or negative) to every note-on velocity, clamped to the legal range, a quick dynamics offset.

└ **VelocityClip** [DYN] [MIDI] in: Audio out: MIDI

Velocity clip: hard-limits note-on velocities to a floor and ceiling, clamping anything outside the window to the edges (a dynamic gate/limiter).

└ **VelocityCompress** [DYN] [MIDI] in: Audio out: MIDI

Velocity compress: pulls every note-on velocity toward the centre by the set ratio, narrowing the dynamic range so soft and loud notes sit closer together.

└ **VelocityCurve** [DYN] [MIDI] in: Audio out: MIDI

Velocity curve: reshapes note-on velocity through a gamma curve - values above 1 soften the response (more playing in the quiet range), below 1 harden it.

└ **VelocityDrift**_[DYN]_[MIDI] in: Audio out: MIDI

Velocity drift: applies a slow bounded random walk to note-on velocities so the dynamics wander gently up and down over time rather than jumping; Step sets the walk speed.

└ **VelocityExpand**_[DYN]_[MIDI] in: Audio out: MIDI

Velocity expand: pushes velocities away from the mid-level (the opposite of compression), so soft notes get softer and loud notes louder - widening the dynamic range of a flatly-played or over-quantized part.

└ **VelocityFloor**_[DYN]_[MIDI] in: Audio out: MIDI

Velocity floor: lifts any note played softer than the floor up to it, guaranteeing every note speaks at a minimum level - a reverse noise gate for taming ghost notes and uneven controllers.

└ **VelocityFold**_[DYN]_[MIDI] in: Audio out: MIDI

Velocity fold: folds note-on velocities that exceed a ceiling back downward (a wavefolder for dynamics), so very hard hits read softer in a non-monotonic way.

└ **VelocityFromInterval**_[DYN]_[MIDI] in: Audio out: MIDI

Velocity from interval: accents each note by how far it leaps from the previous note, so large melodic jumps hit harder than stepwise motion; Depth blends with the played velocity.

└ **VelocityFromPitch**_[DYN]_[MIDI] in: Audio out: MIDI

Velocity from pitch: derives velocity from the note's pitch so higher notes play louder (or, inverted, lower notes louder), a keyboard-tilt dynamic; Depth blends with the played velocity.

└ **VelocityGammaCurve**_[DYN]_[MIDI] in: Audio out: MIDI

Velocity gamma curve: reshapes velocity through a gamma transfer curve - values below 1 lift soft notes and compress dynamics, above 1 push them down and expand - a smooth nonlinear dynamics bend.

└ **VelocityGate**_[DYN]_[MIDI] in: Audio out: MIDI

Velocity gate: passes only notes whose velocity falls inside a Low..High window, masking soft or loud notes (a dynamics filter / range splitter); Invert mutes the window instead of keeping it.

└ **VelocityLFO**_[DYN]_[MIDI] in: Audio out: MIDI

Velocity LFO: sweeps velocity up and down with a slow sine over a set number of notes, an automatic dynamic swell-and-fade that breathes life into a flat sequence; Depth sets how deep the swell.

└ **VelocityMultiply**_[DYN]_[MIDI] in: Audio out: MIDI

Velocity multiply: scales every note-on velocity by a fixed gain factor, a simple dynamics trim that boosts or attenuates the whole part.

└ **VelocityQuantize** [DYN] [MIDI] in: Audio out: MIDI

Velocity quantize: snaps note-on velocities to a small number of evenly-spaced levels, a terraced/stepped dynamic reminiscent of early samplers and harpsichords.

└ **VelocityRandomScale** [DYN] [MIDI] in: Audio out: MIDI

Velocity random scale: multiplies each note's velocity by an independent random gain, scattering the dynamics for a more human, less machine-perfect feel (timing untouched, unlike Humanize); Amount sets the scatter width.

└ **VelocityRandomWalk** [DYN] [MIDI] in: Audio out: MIDI

Velocity random walk: nudges velocity by a smoothed, bounded random walk so the dynamics drift and breathe over time rather than jumping independently each note, a gradual humanizing of touch.

└ **VelocitySmooth** [DYN] [MIDI] in: Audio out: MIDI

Velocity smooth: glides each note's velocity toward a running average of the recent ones, ironing out wild dynamic jumps into an even, controlled performance; Amount sets how strongly it pulls toward the average.

└ **VelocitySwap** [DYN] [MIDI] in: Audio out: MIDI

Velocity swap: inverts each note-on velocity (128 minus the value) so the softest hits become the loudest and vice versa; Depth blends with the played velocity.

└ **VelStep** [DYN] [MIDI] in: Audio out: MIDI

Velocity step: quantizes velocity to a small number of levels (a dynamics bit-crush), flattening expressive playing into stepped terraces from gentle 8-level to brutal 2-level.

└ **WythoffVelocity** [DYN] [MIDI] in: Audio out: MIDI

Wythoff velocity: drives velocity from the lower Wythoff sequence floor($n \cdot \phi$) (the golden Beatty sequence, the winning positions of Wythoff's game: 1,3,4,6,8,9,11,...); Depth blends with the played velocity.

└ **ZeckendorfVelocity** [DYN] [MIDI] in: Audio out: MIDI

Zeckendorf velocity: accents each note by how many Fibonacci numbers it takes to sum to the note count (its Zeckendorf representation length), a number-theoretic accent that grows in golden steps; Depth blends with the played velocity.

└ **ZipfVelocity** [DYN] [MIDI] in: Audio out: MIDI

Zipf velocity: draws each note's velocity from a Zipf (1/rank) distribution, so a few dynamic levels dominate and loud accents are rare - the power-law dynamics of natural performance; Depth blends it against the played velocity.

└ **ZoneVelocityScale** [DYN] [MIDI] in: Audio out: MIDI

Zone velocity scale: scales the velocity of notes inside [Low,High] by Gain, a per-zone dynamics trim that lets a split layer sit louder or softer than the rest.

Expression (3)

☐ **Fall** [FX] [MIDI] in: Audio out: MIDI

Jazz fall-off: on release the pitch sweeps down by Depth over Time while the note rings, then the note releases and the bend snaps back to centre. The note-off is delayed to the end of the fall.

☐ **MidiVibrato** [FX] [MIDI] in: Audio out: MIDI

A pitch-bend LFO that fades in after an Onset delay while any note is held - the delayed, played-in vibrato of a string or voice rather than an instant wobble. Rate in Hz, Depth as bend amount.

☐ **Scoop** [FX] [MIDI] in: Audio out: MIDI

Bends up into each struck note from Depth below, resolving to pitch over Time - the vocal/brass scoop, a pitch-bend ramp that ends at centre so no bend is left hanging.

Harmony (51)

☒ **AutoInvert** [HRM] [MIDI] in: Audio out: MIDI

Cycles the inversion of each struck chord: every trigger rotates which of the lowest notes jump an octave (by Rotate), so a repeated chord keeps climbing through its inversions - movement ChordVoicing's static shapes don't give.

☒ **Bassline** [HRM] [MIDI] in: Audio out: MIDI

Follows the chord with a mono bass voice: the lowest held note dropped Octaves down, retriggered whenever the lowest note changes. Passes the original notes through and adds the bass beneath them.

☒ **Chord** [HRM] [MIDI] in: Audio out: MIDI

Turns each incoming note into a chord (165 types) with inversion, spread, octave doubling and strum.

☒ **ChordLock** [HRM] [MIDI] in: Audio out: MIDI

Quantizes every note to the nearest tone of a fixed chord (Root + Type), tighter than Scale's scale-quantize - locks an improvisation to a chord's notes.

☒ **ChordMem** [HRM] [MIDI] in: Audio out: MIDI

Each key plays a stored chord shape (semitone intervals from node config).

☒ **ChordProg** [HRM] [MIDI] in: Audio out: MIDI

Each note triggers the next chord in a stored root progression (node config).

☒ **ChordVoicing** [HRM] [MIDI] in: Audio out: MIDI

Re-voices each note into a jazz shape (open / shell / quartal / spread).

⚙️ **ConsonanceGate** [HRM] [MIDI] in: Audio out: MIDI

Consonance gate: passes a note only when its interval from the previous one is consonant (unison, third, fourth, fifth, sixth or octave), filtering a chromatic line down to smooth, dissonance-free melodic motion.

⚙️ **ContourGate** [HRM] [MIDI] in: Audio out: MIDI

Contour gate: passes only the rising moves (or only the falling moves) of a melody relative to the previous kept note, isolating the ascending or descending gestures of a line.

⚙️ **ContourReverseGate** [HRM] [MIDI] in: Audio out: MIDI

Contour-reverse gate: passes a note only when the melody changes direction (a peak or valley), keeping just the turning points of a line and dropping the runs between them; Invert keeps the runs.

⚙️ **Counterpoint** [HRM] [MIDI] in: Audio out: MIDI

Adds a second voice in contrary motion - when the melody rises the counter voice falls and vice-versa - quantised to Root/Scale. Interval sets the starting separation.

⚙️ **Diatonic** [HRM] [MIDI] in: Audio out: MIDI

Transposes by scale DEGREES within Root/Scale (a third, a fifth...) instead of by raw semitones like Transpose - the shift always lands back in key. Mod Amt adds a modulatable degree offset.

⚙️ **DiatonicChord** [HRM] [MIDI] in: Audio out: MIDI

Auto-harmoniser: builds the in-key chord on every note by stacking diatonic thirds (triad, 7th...) along Root/Scale, so single-finger melodies become correct chords in the key.

⚙️ **DiatonicHarmonize** [HRM] [MIDI] in: Audio out: MIDI

Diatonic harmonize: adds a second voice a chosen number of scale steps above each note within a major key, so the harmony note always stays in key (true diatonic thirds/sixths, unlike a fixed chromatic interval).

⚙️ **DiatonicTranspose** [HRM] [MIDI] in: Audio out: MIDI

Diatonic transpose: shifts every note by a number of whole scale degrees within a major key, so the melody moves up or down the staff while always landing on in-key notes - true in-key transposition, unlike the chromatic kind.

⚙️ **Drone** [HRM] [MIDI] in: Audio out: MIDI

Doubles every note with a fixed pedal root.

⚙️ **GhostNote** [HRM] [MIDI] in: Audio out: MIDI

Ghost note: doubles every note with a quiet copy a set interval below (default an octave), at a fraction of the original velocity - the subliminal low reinforcement drummers and bassists use to thicken a line.

⌵ **Glissando** [HRM] [MIDI] in: Audio out: MIDI

Fills a stepwise run between consecutive notes.

⌵ **GravityQuantize** [HRM] [MIDI] in: Audio out: MIDI

Gravity quantize: pulls every note toward a center pitch by an adjustable strength, as if the keyboard had a gravitational well - gentle settings nudge stray notes inward, strong settings collapse a whole part toward one tone.

⌵ **HarmonicQuantize** [HRM] [MIDI] in: Audio out: MIDI

Harmonic quantize: re-pitches each note to the nearest member of the natural harmonic series above a root, snapping a chromatic part onto the pure, stretched-then-compressed intervals of the overtone series.

⌵ **Harmonize** [HRM] [MIDI] in: Audio out: MIDI

Adds up to three fixed-interval harmony voices to each note.

⌵ **IntervalClassGate** [HRM] [MIDI] in: Audio out: MIDI

Interval-class gate: passes a note only when its leap from the previous kept note belongs to a chosen interval class (0 unison/octave .. 6 tritone), filtering a line down to one harmonic colour of motion; Invert keeps the rest.

⌵ **Invert** [HRM] [MIDI] in: Audio out: MIDI

Mirrors pitch around a pivot note.

⌵ **LeapGate** [HRM] [MIDI] in: Audio out: MIDI

Leap gate: passes only stepwise motion (intervals below the threshold) or only leaps (intervals at or above it) relative to the previous kept note, splitting a melody into its smooth and its jagged motion.

⌵ **MidiCluster** [HRM] [MIDI] in: Audio out: MIDI

Adds symmetric tone-cluster voices above and below each note.

⌵ **MidiFold** [HRM] [MIDI] in: Audio out: MIDI

Folds out-of-range notes back inside a Low..High window.

⌵ **MirrorPivot** [HRM] [MIDI] in: Audio out: MIDI

Mirror pivot: reflects every note about a fixed pivot pitch ($out = 2 * Pivot - note$), turning rising lines into falling ones and inverting every interval around the chosen axis - melodic inversion with a movable mirror.

⚙️ **ModalShift** [HRM] [MIDI] in: Audio out: MIDI

Modal shift: brightens or darkens the key by raising the fourth toward Lydian or lowering the third, sixth and seventh toward natural minor, recolouring a part's mode without retransposing it.

⚙️ **NegativeHarmony** [HRM] [MIDI] in: Audio out: MIDI

Negative harmony: reflects every note about the axis halfway between the tonic and dominant (Ernst Levy's mirror), swapping major for minor and turning a progression into its tonal shadow while keeping the same harmonic function.

⚙️ **NeoRiemann** [HRM] [MIDI] in: Audio out: MIDI

Neo-Riemannian triad transforms: builds a major/minor triad on each note and applies P (parallel), L (leading-tone) or R (relative) - the maximally-smooth chord moves of late-Romantic and film harmony, each sharing two common tones with the input.

⚙️ **Octave** [HRM] [MIDI] in: Audio out: MIDI

Doubles each note up and/or down by whole octaves.

⚙️ **OctaveClamp** [HRM] [MIDI] in: Audio out: MIDI

Octave clamp: folds every incoming note into the single octave starting at Center by collapsing its pitch class, flattening wide-ranging input into one register - a pitch-class monitor or tight-range re-voicer.

⚙️ **OctaveFoldGate** [HRM] [MIDI] in: Audio out: MIDI

Octave fold: collapses every note into a single octave above a base note (keeping only its pitch class), squashing a wide-ranging part into one register - useful for bass-monitoring or extreme register clamping.

⚙️ **OctaveStackAdd** [HRM] [MIDI] in: Audio out: MIDI

Octave stack: layers each note with a copy an octave above and/or below, fattening a thin line into a multi-octave stack (organ-style); choose which octaves to add.

⚙️ **OvertoneStack** [HRM] [MIDI] in: Audio out: MIDI

Overtone stack: layers quieter harmonic-series voices (octave, octave+fifth, two octaves) above each note with a geometric Rolloff, thickening a single note toward an organ-like harmonic chord.

⚙️ **PitchClassGate** [HRM] [MIDI] in: Audio out: MIDI

Pitch-class gate: passes only notes whose pitch class belongs to the chosen Root and Mode set (chromatic, major, minor, pentatonic or whole-tone), filtering a stream down to a key without transposing anything.

⚙️ **PitchMirror** [HRM] [MIDI] in: Audio out: MIDI

Pitch mirror: reflects every note around a fixed axis pitch, so ascending lines become descending and the melody is turned upside-down about a chosen centre (a literal pitch inversion, key-independent).

⚙️ **PowerChordAdd** [HRM] [MIDI] in: Audio out: MIDI

Power chord: doubles every note with a perfect fifth above (and optionally the octave on top), instantly turning a single line into the root-fifth power chords of rock guitar.

⚙️ **RangeCompress** [HRM] [MIDI] in: Audio out: MIDI

Range compress: linearly squeezes the played pitch range toward a center note, narrowing a wide-ranging part into a tighter register (or, near ratio 1, leaving it alone) - a melodic-ambitus compressor.

⚙️ **RangeExpand** [HRM] [MIDI] in: Audio out: MIDI

Range expand: stretches the played pitch range away from a center note (the inverse of range-compress), exaggerating a cramped melody into a wider, more dramatic ambitus; near ratio 1 it leaves the part alone.

⚙️ **RangeGate** [HRM] [MIDI] in: Audio out: MIDI

Range gate: passes only notes whose pitch lies inside a Low..High window, masking a register in or out of the part; Invert mutes the window instead of keeping it.

⚙️ **ReflectOctave** [HRM] [MIDI] in: Audio out: MIDI

Reflect octave: mirrors each note's pitch class about a pivot while keeping it in its own octave, inverting the melodic contour within every register at once - octave-preserving interval inversion.

⚙️ **RepeatedNoteGate** [HRM] [MIDI] in: Audio out: MIDI

Repeated-note gate: drops a note-on when it repeats the previous kept pitch, passing only when the melody actually changes note - a de-stutter that removes hammered repeats; Invert keeps only the repeats.

⚙️ **Scale** [HRM] [MIDI] in: Audio out: MIDI

Quantizes incoming notes to the nearest degree of the chosen root and scale.

⚙️ **SubHarmonicAdd** [HRM] [MIDI] in: Audio out: MIDI

Sub-harmonic add: reinforces every note with undertone voices an octave (Sub1) and an octave-plus-fifth (Sub2) below at adjustable levels - the subharmonic weight of a Trautonium or organ bourdon.

⚙️ **SubharmonicShift** [HRM] [MIDI] in: Audio out: MIDI

Subharmonic shift: drops each note to the nearest undertone (subharmonic) of a ceiling pitch, the mirror of the harmonic series, building dark, minor-tilted lines beneath an upper anchor note.

⚙️ **Transpose** [HRM] [MIDI] in: Audio out: MIDI

Shifts every note by a fixed semitone amount plus a modulatable offset.

🎵 **TwelveToneRow** [HRM] [MIDI] in: Audio out: MIDI

Twelve-tone row: enforces Schoenberg's serial rule - no pitch class may sound again until all twelve have been played - dropping early repeats so the stream is forced into complete chromatic rows.

🎵 **UnisonDetuneAdd** [HRM] [MIDI] in: Audio out: MIDI

Unison detune: layers a second copy of each note slightly detuned (via per-note microtuning), thickening a mono line into a wide chorused unison without a separate effect.

🎵 **VoiceLead** [HRM] [MIDI] in: Audio out: MIDI

Smooth voice leading: places each note's pitch class in the octave nearest the previous output, so a line or chord progression moves by the smallest interval. Range caps the leap; past it the original octave is kept.

🎵 **WholeToneSnap** [HRM] [MIDI] in: Audio out: MIDI

Whole-tone snap: quantizes every note to the nearest degree of the six-note whole-tone scale from Root, dissolving tonality into the dreamy, rootless ambiguity that scale is famous for.

Modulation (60)

🎵 **AvgPitchCC** [MOD] [MIDI] in: Audio out: MIDI

Average-pitch CC: emits a controller that follows the mean pitch of all held notes (the chord's centroid), a smooth register-position modulation source.

🎵 **BendFromVelocity** [MOD] [MIDI] in: Audio out: MIDI

Bend from velocity: emits a pitch-bend on each note-on scaled by how hard you played, so harder hits bend up further (an expressive velocity-to-pitch gesture); Depth sets the range.

🎵 **BluesInflect** [MOD] [MIDI] in: Audio out: MIDI

Blues inflect: bends the third, fifth and seventh of a major key downward toward the blue notes (the cracks between the piano keys that a blues singer or guitarist bends into), adding microtonal blues feel without changing the notes played.

🎵 **BohlenPierceTune** [MOD] [MIDI] in: Audio out: MIDI

Bohlen-Pierce tune: retunes the keyboard to the Bohlen-Pierce scale, which divides not the octave but the 3:1 'tritave' into 13 equal steps, giving an alien but surprisingly consonant tuning built on odd-harmonic ratios.

🎵 **ByzantineTune** [MOD] [MIDI] in: Audio out: MIDI

Byzantine tune: retunes to a Byzantine chant scale, whose soft-chromatic genus uses neutral intervals absent from Western tuning, the sound of Eastern Orthodox liturgical melody.

🎵 **CarlosAlphaTune** [MOD] [MIDI] in: Audio out: MIDI

Carlos Alpha tune: retunes to Wendy Carlos's Alpha scale of 78-cent equal steps, a non-octave temperament engineered to nail pure major thirds and fifths at the cost of ever repeating at the octave - a shimmering, otherworldly intonation.

🎵 **CarlosBetaTune** [MOD] [MIDI] in: Audio out: MIDI

Carlos Beta tune: retunes to Wendy Carlos's Beta scale, built from a ~63.8-cent step that deliberately does not repeat at the octave, giving the uncanny non-octave harmony of her Beauty in the Beast.

🎵 **CarlosGammaTune** [MOD] [MIDI] in: Audio out: MIDI

Carlos Gamma tune: retunes to Wendy Carlos's Gamma scale, an ultra-fine ~35.1-cent micro-step tuning that renders both perfect fifths and major thirds nearly pure, the smoothest of her invented scales.

🎵 **CCGlide** [MOD] [MIDI] in: Audio out: MIDI

Portamento for a CC: smooths the watched CC toward each newly received value over Time, emitting interpolated CC each block - de-zippers a stepped controller, where MidiSlew only glides the internal scalar.

🎵 **CCLFO** [MOD] [MIDI] in: Audio out: MIDI

Continuous CC LFO generator (sine/triangle/saw/square) emitted on a chosen CC at a free Rate - automates a synth parameter over MIDI, where MidiLFO drives only the internal mod-matrix scalar.

🎵 **CCSeq** [MOD] [MIDI] in: Audio out: MIDI

Tempo-synced step CC sequencer: one CC value per synced step cycling an 8-step shape (overridable via the node's config list) - a modulation sequencer in the CC domain, where StepSeqMidi sequences notes.

🎵 **ChordSpanCC** [MOD] [MIDI] in: Audio out: MIDI

Chord-span CC: emits a controller that reflects the interval span (highest minus lowest) of the held notes, full at Range semitones - wide voicings push it up.

🎵 **ExpressionSwell** [MOD] [MIDI] in: Audio out: MIDI

Expression swell: emits a CC11 expression value that ramps up from zero after each note-on over the swell time, an automatic volume fade-in / crescendo per note for strings and pads.

🎵 **GamelanDetune** [MOD] [MIDI] in: Audio out: MIDI

Gamelan detune: tunes alternate keys slightly apart so paired notes shimmer with the slow acoustic beating of a gamelan's deliberately-mistuned instrument pairs (ombak) - an ensemble-wide chorused waver.

🎵 **Generative** [MOD] [MIDI] in: Audio out: MIDI

Probabilistic melodic random walk synced to a rate.

🎵 **GoldenAngleDetune** [MOD] [MIDI] in: Audio out: MIDI

Golden-angle detune: scatters each successive note's microtuning by the golden angle, the maximally-even irrational spacing, so repeated notes never land on the same detune - an organic, never-repeating pitch shimmer.

🎵 **HarmonicSeriesSnap** [MOD] [MIDI] in: Audio out: MIDI

Harmonic-series snap: detunes each note to the nearest member of the natural harmonic series above a root, replacing tempered intervals with the pure, beatless ratios of just intonation as a brass or didgeridoo would sound them.

🎵 **HarmonicSeriesTune** [MOD] [MIDI] in: Audio out: MIDI

Harmonic-series tune: retunes every note to the nearest member of the natural overtone series of a root (the just-intonation pitches a vibrating string actually produces), so chords lock into pure, beatless ratios that no equal scale can reach.

🎵 **HighNoteCC** [MOD] [MIDI] in: Audio out: MIDI

High-note CC: emits a controller that tracks the pitch of the highest held note, so the top voice of your playing steers a synth parameter (filter, level, ...).

🎵 **HirajoshiTune** [MOD] [MIDI] in: Audio out: MIDI

Hirajoshi tune: retunes to the Japanese hirajoshi pentatonic, the dark, koto-flavoured five-note scale of traditional Japanese music, snapped from the played notes.

🎵 **IntervalCC** [MOD] [MIDI] in: Audio out: MIDI

Interval CC: emits a controller on each note-on from how far it leaps from the previous note, so stepwise lines read low and big jumps read high, full at Range semitones.

🎵 **JustMinorTune** [MOD] [MIDI] in: Audio out: MIDI

Just-minor tune: retunes to a 5-limit just-intonation natural minor (pure 6/5 third and 3/2 fifth), giving beatless minor chords impossible in equal temperament.

🎵 **LowNoteCC** [MOD] [MIDI] in: Audio out: MIDI

Low-note CC: emits a controller that tracks the pitch of the lowest held note, so the bass voice of your playing steers a synth parameter.

🎵 **MaqamRastTune** [MOD] [MIDI] in: Audio out: MIDI

Maqam Rast tune: retunes to the Arabic maqam Rast, whose neutral third and sixth sit a quarter-tone below the Western major, giving the characteristic in-between intervals of Middle-Eastern melody.

🎵 **MeantoneTune** [MOD] [MIDI] in: Audio out: MIDI

Meantone tune: retunes to quarter-comma meantone, the Renaissance/Baroque temperament with pure (beatless) major thirds bought at the price of a howling 'wolf' fifth, the sound of early-music keyboards.

🎵 **MicrotuneSpread** [MOD] [MIDI] in: Audio out: MIDI

Microtune spread: gives every note a small random pitch detune up to +/-Cents via its microtuning offset, loosening rigid tuning into a chorused, slightly-out ensemble shimmer.

🎵 **MidiEnv** [MOD] [MIDI] in: Audio out: MIDI

ADSR envelope triggered by the note gate, on the scalar control output.

🎵 **MidiLFO** [MOD] [MIDI] in: Audio out: MIDI

Low-frequency modulation source (sine/tri/saw/square) on the scalar control output.

🎵 **MidiRandom** [MOD] [MIDI] in: Audio out: MIDI

Generates random notes within a pitch range at a synced rate.

🎵 **MidiSampleHold** [MOD] [MIDI] in: Audio out: MIDI

Samples and holds the incoming control value on each note.

🎵 **MidiSlew** [MOD] [MIDI] in: Audio out: MIDI

Smooths (glides) the control value over a set time.

🎵 **NEdoRetune** [MOD] [MIDI] in: Audio out: MIDI

N-EDO retune: detunes every note from 12-tone equal temperament to the nearest step of an N-equal-divisions-of-the-octave scale, opening up 19, 24, 31 or any microtonal grid without changing the played note numbers.

🎵 **NoteOnTrigCC** [MOD] [MIDI] in: Audio out: MIDI

Note-on trigger CC: fires a CC pulse that jumps to full on every note-on and decays away, a built-in trigger envelope you can route to a synth parameter; Decay sets the fall time.

🎵 **NoteToPressure** [MOD] [MIDI] in: Audio out: MIDI

Note-to-pressure: emits a channel-aftertouch value scaled from each note's velocity, so how hard you strike also drives any pressure-mapped destination (vibrato, filter, swell) - turning a non-aftertouch keyboard into an expressive one.

🎵 **PartchTune** [MOD] [MIDI] in: Audio out: MIDI

Partch tune: retunes to an 11-limit just-intonation subset in the spirit of Harry Partch's 43-tone tonality, the ratio-based microtonality of his hand-built instruments.

🎵 **PeLogTune** [MOD] [MIDI] in: Audio out: MIDI

PeLog tune: retunes to a Javanese pelog scale - seven unequal steps with characteristic narrow and wide intervals - the more tense, expressive counterpart to slendro in gamelan music.

🎵 **PersianTune** [MOD] [MIDI] in: Audio out: MIDI

Persian tune: retunes to a Persian dastgah scale with its characteristic koron (quarter-flat) degrees, the microtonal intervals of classical Persian music played on the tar and santur.

🎵 **PitchBendLFO** [MOD] [MIDI] in: Audio out: MIDI

Pitch-bend LFO: emits a free-running pitch-bend sine that wobbles the tuning continuously whether or not notes are playing - a global vibrato/auto-bend source independent of any single note; Rate and Depth shape the sweep.

🎵 **PitchClassCC** [MOD] [MIDI] in: Audio out: MIDI

Pitch-class CC: emits a controller on each note-on from the note's pitch class (its position 0-11 within the octave, octave-independent), a harmonic-colour modulation source.

🎵 **PitchDrift** [MOD] [MIDI] in: Audio out: MIDI

Nudges each note's pitch by a small random detune up to +/-Cents (per-note, polyphonic) - the gentle oscillator instability of analog gear, applied to pitch where Humanize only scatters timing and velocity.

🎵 **PitchScoop** [MOD] [MIDI] in: Audio out: MIDI

Pitch scoop: emits a pitch-bend that starts bent below each new note and swoops up to true pitch over the scoop time, the expressive slide-into-note of a fretless or vocal attack.

🎵 **PolyCountCC** [MOD] [MIDI] in: Audio out: MIDI

Poly-count CC: emits a controller that reflects how many notes are held (chord density), reaching full value at Max voices - play thicker chords to open a parameter.

🎵 **PressSwell** [MOD] [MIDI] in: Audio out: MIDI

Generates a channel-pressure (aftertouch) swell that rises to Max over Rise while any note is held, snapping back to zero on release - an automatic crescendo for pads and strings, where ATProc only transforms incoming aftertouch.

🎵 **PythagoreanTune** [MOD] [MIDI] in: Audio out: MIDI

Pythagorean tune: retunes to 3-limit Pythagorean intonation built from a spiral of pure fifths, giving brilliant, wide major thirds and the bright, slightly-sharp leading tones of medieval and string-ensemble tuning.

🎵 **QuarterToneShift** [MOD] [MIDI] in: Audio out: MIDI

Quarter-tone shift: detunes every note by up to a quarter-tone via its microtuning offset, sliding the whole performance off the 12-tone grid into 24-tone territory without changing the note numbers.

🎵 **QuarterToneTune** [MOD] [MIDI] in: Audio out: MIDI

Quarter-tone tune: retunes to 24-tone equal temperament, snapping every note to the nearest 50-cent quarter-tone, the standard grid of Arabic maqam notation and much 20th-century microtonal music.

~ **RagaBhairavTune** [MOD] [MIDI] in: Audio out: MIDI

Raga Bhairav tune: retunes to the just-intonation shrutis of the North-Indian morning raga Bhairav (with its flat second and flat sixth), bringing the pure, beatless intervals of Indian classical tuning.

~ **RandOctave** [MOD] [MIDI] in: Audio out: MIDI

Randomly shifts notes by up to +/- Range octaves with a chance.

~ **RandomCC** [MOD] [MIDI] in: Audio out: MIDI

Random CC: fires a random value on a chosen controller number with every note-on, an instant source of evolving per-note modulation (filter, pan, anything) that never repeats; Amount sets the spread around center.

~ **SeptimalTune** [MOD] [MIDI] in: Audio out: MIDI

Septimal tune: retunes to a 7-limit just scale including the harmonic seventh (7/4), the deep, locked-in dominant-seventh sound prized in barbershop and blues harmony.

~ **SlendroTune** [MOD] [MIDI] in: Audio out: MIDI

Slendro tune: retunes the keyboard to a Javanese gamelan slendro scale - five near-equal steps spanning the octave - giving the floating, anchorless quality of Indonesian slendro tuning, snapped from the nearest played note.

~ **StretchTuning** [MOD] [MIDI] in: Audio out: MIDI

Stretch tuning: applies a Railsback-style octave stretch, tuning notes progressively sharp as they rise and flat as they fall, the psychoacoustic tuning real pianos use so octaves sound in tune to the ear.

~ **SubharmonicTune** [MOD] [MIDI] in: Audio out: MIDI

Subharmonic tune: retunes every note to the nearest member of the undertone (subharmonic) series descending from a top pitch, the mirror image of the overtone series that gives eerie, hollow, just-intoned minor sonorities.

~ **SwellPressure** [MOD] [MIDI] in: Audio out: MIDI

Swell pressure: emits a channel-pressure that rises the longer notes are held and falls once they release, an automatic aftertouch swell for sustained pads; Rate sets the speed.

~ **ThaiTune** [MOD] [MIDI] in: Audio out: MIDI

Thai tune: retunes to the Thai 7-tone equal temperament (seven equal steps to the octave), the tuning of the Thai ranat and pi phat ensemble that sits between the Western diatonic pitches.

~ **TurkishMakamTune** [MOD] [MIDI] in: Audio out: MIDI

Turkish makam tune: retunes to a Turkish makam built on Holdrian (53-comma) steps, whose microtonal neutral seconds and sevenths give the characteristic intervals of Ottoman classical melody.

🎵 **VelToCC** [MOD] [MIDI] in: Audio out: MIDI

Turns each note's velocity into a CC value emitted just before the note, so a synth's filter or level opens with how hard you played - dynamics routed to expression, where Note2CC sends pitch.

🎵 **VelToDetune** [MOD] [MIDI] in: Audio out: MIDI

Velocity-to-detune: maps how hard each note is played to a microtuning detune, so dynamics bend pitch slightly sharp or flat - the pitch-pulls-with-force behaviour of acoustic instruments and analog drift.

🎵 **WerckmeisterTune** [MOD] [MIDI] in: Audio out: MIDI

Werckmeister tune: retunes to Werckmeister III, the famous 1691 well-temperament whose unequal fifths give each key its own colour (the 'well-tempered' tuning of Bach's era) - subtle, key-dependent detuning from equal temperament.

🎵 **YoungTune** [MOD] [MIDI] in: Audio out: MIDI

Young tune: retunes to Thomas Young's 1799 well-temperament, where each key has its own subtly different colour - near-pure common keys, spicier remote ones - the way Classical-era keyboards actually sounded.

Probability (180)

🎲 **AbundantGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Abundant gate: passes note-ons only on abundant counts - numbers whose proper divisors sum to more than themselves (12, 18, 20, 24...) - thinning the stream onto the divisor-rich integers; Invert keeps the deficient ones.

🎲 **AchillesGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Achilles gate: passes note-ons on Achilles counts - powerful numbers (every prime factor squared) that are not themselves a perfect power (72, 108, 200, 288, ...); Invert keeps the rest.

🎲 **AdditivePrimeGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Additive-prime gate: passes note-ons on additive-prime counts that are prime and whose digit sum is also prime (2, 3, 5, 7, 11, 23, 29, 41, 43, ...); Invert keeps the rest.

🎲 **AlmostPrimeGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Almost-prime gate: passes note-ons on k-almost-prime counts having exactly K prime factors counted with multiplicity (K=1 primes, K=2 semiprimes, K=3 ...), one knob spanning the whole almost-prime family; Invert keeps the rest.

🎲 **AntiprimeGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Antiprime gate: passes note-ons on highly-composite (antiprime) counts that have more divisors than every smaller number (1, 2, 4, 6, 12, 24, 36, 48, 60, ...), the most-divisible records; Invert keeps the rest.

⚙️ **AntiRepeatGate** [PRB][CV][MIDI] in: Audio out: MIDI

Anti-repeat gate: drops a note when it repeats the immediately-previous pitch, forcing the line to keep moving instead of hammering one note - an automatic de-stutter for melodic variety.

⚙️ **AscendingDigitGate** [PRB][CV][MIDI] in: Audio out: MIDI

Ascending-digit gate: passes note-ons whose decimal digits strictly increase from left to right (1..9,12,13,...,123,124,...); Invert keeps the rest.

⚙️ **AutomorphicGate** [PRB][CV][MIDI] in: Audio out: MIDI

Automorphic gate: passes note-ons on automorphic counts whose square ends in the number itself (5,6,25,76,376,625,...), a curious self-reproducing-under-squaring set; Invert keeps the rest.

⚙️ **BellNumberGate** [PRB][CV][MIDI] in: Audio out: MIDI

Bell-number gate: passes note-ons on Bell-number counts (1,2,5,15,52,203,877,...), which count the ways to partition a set into groups; their fast growth thins the stream quickly. Invert keeps the rest.

⚙️ **BinaryPalindromeGate** [PRB][CV][MIDI] in: Audio out: MIDI

Binary-palindrome gate: passes note-ons whose count reads the same in binary forwards and backwards (1,3,5,7,9,15,17,21,...), a self-mirroring bit pattern distinct from decimal palindromes; Invert keeps the rest.

⚙️ **BinaryWeightGate** [PRB][CV][MIDI] in: Audio out: MIDI

Binary-weight gate: passes note-ons whose count has exactly the chosen number of 1-bits in binary (its Hamming weight), a bit-pattern sieve that thins to sparser, more clustered hits as the weight rises; Invert keeps the rest.

⚙️ **BlumIntegerGate** [PRB][CV][MIDI] in: Audio out: MIDI

Blum-integer gate: passes note-ons on Blum counts that are the product of two distinct primes both congruent to 3 mod 4 (21,33,57,69,77,...), the moduli behind Blum-Blum-Shub cryptography; Invert keeps the rest.

⚙️ **CakeNumberGate** [PRB][CV][MIDI] in: Audio out: MIDI

Cake-number gate: passes note-ons on cake counts (the most pieces a cake splits into with k planar cuts: 1,2,4,8,15,26,42,...), the 3-D analogue of the lazy-caterer numbers; Invert keeps the rest.

⚙️ **CarmichaelGate** [PRB][CV][MIDI] in: Audio out: MIDI

Carmichael gate: passes note-ons on Carmichael counts - squarefree composites that fool the Fermat primality test (561,1105,1729,...) - via Korselt's criterion; an extremely sparse set. Invert keeps the rest.

⚙️ **CatalanGate** [PRB][CV][MIDI] in: Audio out: MIDI

Catalan gate: passes note-ons only on Catalan-number counts (1,2,5,14,42,...), the combinatorial sequence counting trees and bracketings, which thins out fast as the values explode; Invert keeps the others.

 **CatalanParityGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Catalan-parity gate: passes note-ons when the Catalan number $C(\text{count})$ is odd - which happens only when the count-plus-one is a power of two - giving a sparse, exponentially-spaced pattern; Invert keeps the rest.

 **CenteredCubeGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Centered-cube gate: passes note-ons on centered-cube counts (1,9,35,91,189,...), the dots in a cube grown shell by shell around a center; Invert keeps the rest.

 **CenteredDecagonalGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Centered-decagonal gate: passes note-ons on centered-decagonal counts (1,11,31,61,101,...), a ten-sided figure grown ring by ring around a center; Invert keeps the rest.

 **CenteredHexGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Centered-hexagonal gate: passes note-ons on centered-hexagonal counts (1,7,19,37,61,...), the numbers of dots in a growing hexagonal grid (and the rows of Pascal-triangle hex packing); Invert keeps the rest.

 **CenteredNonagonalGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Centered-nonagonal gate: passes note-ons on centered-nonagonal counts (1,10,28,55,91,...), a nine-sided figure grown ring by ring around a center, which are also every third triangular number; Invert keeps the rest.

 **CenteredPentagonalGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Centered-pentagonal gate: passes note-ons on centered-pentagonal counts (1,6,16,31,51,76,...), a pentagon grown ring by ring around a center dot; Invert keeps the rest.

 **CenteredSquareGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Centered-square gate: passes note-ons on centered-square counts (1,5,13,25,41,...), the dots in a square diamond grown ring by ring; Invert keeps the rest.

 **CenteredTriangularGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Centered-triangular gate: passes note-ons on centered-triangular counts (1,4,10,19,31,...), a triangular grid grown outward from a central dot; Invert keeps the rest.

 **Chance** [PRB] [CV] [MIDI] in: Audio out: MIDI

Probabilistic gate: lets each note through with the set probability.

 **CircularPrimeGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Circular-prime gate: passes note-ons on circular-prime counts where every cyclic rotation of the decimal digits is itself prime (2,3,5,7,11,13,17,37,79,...); Invert keeps the rest.

⚙️ **CollatzGate** [PRB][CV][MIDI] in: Audio out: MIDI

Collatz gate: runs the $3n+1$ process on each note's count and passes it when the number of steps to reach 1 is even - an erratic but deterministic gate straight from the unsolved Collatz conjecture; Invert takes the odd-length ones.

⚙️ **CollatzPeakGate** [PRB][CV][MIDI] in: Audio out: MIDI

Collatz-peak gate: passes note-ons whose count sends its Collatz ($3n+1$) trajectory soaring above Mult times the starting value, spotlighting the counts that take the wildest hailstone excursions; Invert keeps the rest.

⚙️ **ContraryGate** [PRB][CV][MIDI] in: Audio out: MIDI

Contrary gate: passes a note only if it moves in the opposite direction to the previous melodic step, forcing the line into constant zig-zag contrary motion and filtering out runs that keep climbing or falling.

⚙️ **CoprimeGate** [PRB][CV][MIDI] in: Audio out: MIDI

Coprime gate: passes a note only when its count shares no common factor with the modulus ($\gcd = 1$), thinning the stream onto the totatives of the modulus - a number-theoretic sieve; Invert keeps the non-coprime counts.

⚙️ **CoprimeStepGate** [PRB][CV][MIDI] in: Audio out: MIDI

Coprime-step gate: passes a note only when its running count shares no common factor with the previous kept count (their $\gcd$ is 1), a number-theoretic thinning that favours relatively-prime spacings; Invert keeps the rest.

⚙️ **CousinPrimeGate** [PRB][CV][MIDI] in: Audio out: MIDI

Cousin-prime gate: passes note-ons on counts in a cousin-prime pair (two primes four apart: 3,7,13,17,19,23,...); Invert keeps the rest.

⚙️ **CubeGate** [PRB][CV][MIDI] in: Audio out: MIDI

Cube gate: passes note-ons on perfect-cube counts (1,8,27,64,125,...), an exponentially-widening figurate spacing; Invert keeps the rest.

⚙️ **CullenGate** [PRB][CV][MIDI] in: Audio out: MIDI

Cullen gate: passes note-ons on Cullen-number counts ($n \cdot 2^{n+1}$: 3,9,25,65,161,385,...), the +1 companions of the Woodall numbers; Invert keeps the rest.

⚙️ **DeficientGate** [PRB][CV][MIDI] in: Audio out: MIDI

Deficient gate: passes note-ons on deficient counts whose proper divisors sum to less than the number (1,2,3,4,5,7,8,9,10,11,...; the majority of integers); Invert keeps the abundant/perfect ones.

⚙️ **DelannoyGate** [PRB][CV][MIDI] in: Audio out: MIDI

Delannoy gate: passes note-ons on central-Delannoy counts (1,3,13,63,321,1683,...), the king-move lattice paths from corner to corner of a grid; Invert keeps the rest.

 **DescendingDigitGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Descending-digit gate: passes note-ons of 2+ digits whose decimal digits strictly decrease from left to right (10,20,21,30,31,32,...); Invert keeps the rest.

 **DigitalRootGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Digital-root gate: passes note-ons whose count has a chosen digital root (the single digit reached by repeatedly summing digits), giving a clean period-9 repeating pattern; Invert keeps the rest.

 **DigitalRootPrimeGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Digital-root-prime gate: passes note-ons whose digital root is a prime digit (2, 3, 5 or 7), a clean period-9 repeating pattern; Invert keeps the rest.

 **DigitCountGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Digit-count gate: passes note-ons whose running count has exactly the chosen number of decimal digits, opening for a block of counts then closing - a coarse decimal-magnitude window; Invert keeps the rest.

 **DigitFactorialGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Factorion gate: passes note-ons on the rare factorion counts equal to the sum of the factorials of their digits (1,2,145,40585); mostly silent, firing only on those four. Invert keeps the rest.

 **DigitGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Digit-sum gate: passes a note unless the decimal digit-sum of its count is divisible by Modulo, carving a self-similar arithmetic rhythm out of a steady stream; Invert keeps only the divisible ones.

 **DigitProductGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Digit-product gate: passes a note only when the product of its count's decimal digits exceeds a threshold, so counts containing a zero or small digits drop out - a digit-driven thinning with a tunable density. Invert keeps the rest.

 **DigitProductPrimeGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Digit-product-prime gate: passes note-ons where the product of the nonzero decimal digits is prime - exactly one prime digit (2,3,5,7) among ones (2,3,5,7,12,13,15,...); Invert keeps the rest.

 **DigitSumGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Digit-sum gate: passes note-ons whose decimal digit sum equals the chosen target, a repeating digit-driven pattern whose density you set with the target; Invert keeps the rest.

⚙️ **DigitSumSquareGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Digit-sum-square gate: passes note-ons whose decimal digit sum is a perfect square (1,4,9,16,...), so counts summing to 1, 4, 9 or 16 pass; Invert keeps the rest.

⚙️ **DivisorCountGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Divisor-count gate: passes note-ons whose count has exactly the chosen number of divisors (tau=2 selects primes, tau=3 prime squares, and so on), a divisor-structure sieve; Invert keeps the rest.

⚙️ **DivisorParityGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Divisor-parity gate: passes note-ons on counts with an odd number of divisors, which are exactly the perfect squares (1,4,9,16,25,...) - a gate that fires on an ever-widening square spacing; Invert keeps the rest.

⚙️ **DoubleFactorialGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Double-factorial gate: passes note-ons on double-factorial counts $n!!$ - the product of integers of the same parity down to 1 (1,2,3,8,15,48,105,384,...); Invert keeps the rest.

⚙️ **DudeneyGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Dudeney gate: passes note-ons on Dudeney counts that are perfect cubes whose digit sum equals the cube root (1,512,4913,5832,17576,19683); an extremely rare set. Invert keeps the rest.

⚙️ **EmirpGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Emirp gate: passes note-ons on emirp counts - primes that turn into a different prime when their digits are reversed (13,17,31,37,71,73,79,...); Invert keeps the rest.

⚙️ **EquidigitalGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Equidigital gate: passes note-ons on equidigital counts that take the same number of digits to write as their prime factorization does (1,2,3,5,7,10,...); Invert keeps the rest.

⚙️ **EvenDigitGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Even-digit gate: passes note-ons whose count is written with only even decimal digits (2,4,6,8,20,22,24,...), a simple digit-pattern thinning; Invert passes counts containing an odd digit.

⚙️ **EvenDigitSumGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Even-digit-sum gate: passes note-ons whose decimal digit sum is even, a near-even-split digit pattern; Invert keeps the odd-digit-sum counts.

⚙️ **ExtravagantGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Extravagant gate: passes note-ons on extravagant counts whose prime factorization takes MORE digits to write than the number itself (4,6,8,9,12,18,...); Invert keeps the rest.

⚙️ **FermatPrimeGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Fermat-prime gate: passes note-ons on the five known Fermat primes ($2^{2^k}+1$: 3,5,17,257,65537), the primes behind constructible polygons; an exceedingly rare set. Invert keeps the rest.

⚙️ **FibGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Fibonacci gate: passes a note unless the Fibonacci number of its ordinal is divisible by Modulo, carving a golden, self-similar rhythm out of a steady stream; Invert keeps the divisible ones.

⚙️ **FibonacciGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Fibonacci gate: passes note-ons on Fibonacci-number counts (1,2,3,5,8,13,21,34,...), the golden-ratio recurrence whose gaps grow geometrically; Invert keeps the rest.

⚙️ **FibonacciPrimeGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Fibonacci-prime gate: passes note-ons on Fibonacci counts that are also prime (2,3,5,13,89,233,1597,...), a doubly-special sparse set; Invert keeps the rest.

⚙️ **FrugalGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Frugal gate: passes note-ons on frugal counts that take fewer digits to write than their prime factorization does (125=5³, 128=2⁷, ...), an economical-number sieve; Invert keeps the rest.

⚙️ **GapGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Gap gate: passes a note only after at least Gap notes have gone by since the last one it let through, a refractory thinner that guarantees a minimum spacing between surviving notes regardless of input density.

⚙️ **GeneralizedPentagonalGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Generalized-pentagonal gate: passes note-ons on generalized-pentagonal counts (1,2,5,7,12,15,22,26,...), the exponents in Euler's pentagonal-number-theorem partition product; Invert keeps the rest.

⚙️ **GoldenGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Golden gate: passes notes by a golden-ratio low-discrepancy sequence against Density, so the kept notes spread far more evenly through time than random chance ever would - a deterministic, clump-free thinning.

⚙️ **GolombGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Golomb gate: passes note-ons on Golomb-sequence counts (1,2,3,4,5,...; the non-decreasing self-describing sequence where term n says how often n appears), a gently-climbing staircase set; Invert keeps the rest.

⚙️ **HammingNumberGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Hamming-number gate: passes note-ons on regular (5-smooth) counts whose only prime factors are 2, 3 and 5 (1,2,3,4,5,6,8,9,10,12,15,16,...), the Hamming/Humble numbers of computing lore; Invert keeps the rest.

⚙️ **HappyGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Happy gate: passes notes on happy-number counts - those whose repeated sum-of-squared-digits reaches 1 - and drops the sad ones that fall into the 4-16-37 cycle; Invert flips which set plays.

⚙️ **HeptanacciGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Heptanacci gate: passes note-ons on heptanacci counts where each term is the sum of the previous seven (1,2,4,8,16,32,64,127,...), approaching a doubling sequence; Invert keeps the rest.

⚙️ **HexagonalPyramidalGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Hexagonal-pyramidal gate: passes note-ons on hexagonal-pyramidal counts (1,7,25,63,129,...), the cannonballs in a hexagon-based pyramid; Invert keeps the rest.

⚙️ **HexanacciGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Hexanacci gate: passes note-ons on hexanacci counts where each term is the sum of the previous six (1,2,4,8,16,32,63,125,...); Invert keeps the rest.

⚙️ **HighlyAbundantGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Highly-abundant gate: passes note-ons on highly-abundant counts whose divisor sum beats that of every smaller number (1,2,3,4,6,8,10,12,16,18,20,24,...); Invert keeps the rest.

⚙️ **HighlyCompositeGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Highly-composite gate: passes note-ons only on counts that set a new record for number of divisors (1,2,4,6,12,24,36,48,60,...), the maximally-factorable integers; a rare, widening-gap accent pulse. Invert keeps the rest.

⚙️ **HoaxNumberGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Hoax-number gate: passes note-ons on hoax counts whose digit sum equals the summed digit sums of their DISTINCT prime factors (22,58,84,...), the squarefree-distinct cousin of Smith numbers; Invert keeps the rest.

⚙️ **HofstadterQGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Hofstadter-Q gate: passes note-ons on values that occur in the chaotic Hofstadter Q meta-sequence ($Q(n)=Q(n-Q(n-1))+Q(n-Q(n-2))$), an erratic self-referential set from Godel, Escher, Bach; Invert keeps the rest.

⚙️ **IntervalGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Interval gate: passes a note only if its leap from the previous kept note lies between Min and Max semitones, filtering a line down to small steps or to wide jumps - a melodic-contour sieve.

⚙️ **JacobsthalGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Jacobsthal gate: passes note-ons on Jacobsthal-number counts (1,3,5,11,21,43,...; $J(n)=J(n-1)+2J(n-2)$), a Fibonacci-like sequence whose ratio tends to 2; Invert keeps the rest.

⚙️ **KaprekarGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Kaprekar gate: passes note-ons on Kaprekar counts whose square can be split into two parts that add back to the number (1,9,45,55,99,297,703,999,...); Invert keeps the rest.

⚙️ **KeithNumberGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Keith-number gate: passes note-ons on Keith (repfigit) counts - numbers that reappear in the Fibonacci-like sequence seeded by their own digits (14,19,28,47,...) - an extremely rare self-referential set; Invert keeps the rest.

⚙️ **LazyCatererGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

lazy-caterer gate: passes note-ons on central-polygonal (lazy-caterer) counts (1,2,4,7,11,16,22,...), the maximum pieces a pancake splits into with n straight cuts; Invert keeps the rest.

⚙️ **LeonardoGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Leonardo gate: passes note-ons on Leonardo-number counts ($L(n)=L(n-1)+L(n-2)+1$: 1,3,5,9,15,25,41,...), the sizes behind Dijkstra's smoothsort heaps; Invert keeps the rest.

⚙️ **LeylandGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Leyland gate: passes note-ons on Leyland-number counts ($x^y \cdot y^x$ for $x,y \geq 2$: 8,17,32,54,57,100,145,...), a sparse two-exponent set; Invert keeps the rest.

⚙️ **LiouvilleGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Liouville gate: passes note-ons when the Liouville function is +1 (the count has an even number of prime factors with multiplicity), an almost-balanced parity stream from analytic number theory; Invert keeps the odd-factor counts.

⚙️ **LucasGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Lucas gate: passes note-ons only on Lucas-number counts (2,1,3,4,7,11,18,...), the Fibonacci companion sequence, thinning the stream onto golden-ratio-spaced ordinals; Invert keeps the rest.

⚙️ **LucasPrimeGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Lucas-prime gate: passes note-ons on Lucas counts that are also prime (2,3,7,11,29,47,199,521,...); Invert keeps the rest.

⚙️ **LuckyNumberGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Lucky-number gate: passes note-ons on Ulam's lucky counts, the survivors of a Josephus-style sieve (1,3,7,9,13,15,21,25,...) that share many properties with the primes; Invert keeps the rest.

⚙️ **MagicConstantGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Magic-constant gate: passes note-ons on magic-square constant counts $n(n^2+1)/2$ - the common row/column/diagonal sum of an $n \times n$ magic square (1,5,15,34,65,111,...); Invert keeps the rest.

⚙️ **MersenneGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Mersenne gate: passes note-ons on Mersenne-number counts (one less than a power of two: 1,3,7,15,31,63,127), the all-ones binary numbers; their exponential spacing thins the stream fast. Invert keeps the rest.

⚙️ **MersennePrimeGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Mersenne-prime gate: passes note-ons on Mersenne-prime counts (2^p-1 that are prime: 3,7,31,127,...), an extremely sparse, exponentially-spaced set tied to the perfect numbers; Invert keeps the rest.

⚙️ **MertensGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Mertens gate: passes note-ons while the Mertens function (the running sum of the Moebius function up to the count) is positive, a slow random-walk-like sign whose growth is tied to the Riemann hypothesis; Invert keeps the negative stretches.

⚙️ **MoebiusGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Moebius gate: passes a note only when its count is squarefree (the Moebius function is non-zero), dropping any note whose ordinal is divisible by a perfect square - a sieve-flavoured aperiodic gate; Invert keeps the square-divisible ones.

⚙️ **MoranGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Moran gate: passes note-ons on Moran counts - Harshad numbers whose quotient (number over its digit sum) is prime (18,21,27,42,45,...); Invert keeps the rest.

⚙️ **MotzkinGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Motzkin gate: passes note-ons on Motzkin-number counts (1,2,4,9,21,51,127,...), the combinatorial sequence counting non-crossing chords and lattice paths; Invert keeps the rest.

⚙️ **MultiplicativePersistenceGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Multiplicative-persistence gate: passes note-ons whose multiplicative persistence - the number of times you replace the count by the product of its digits before hitting a single digit - equals the chosen target; Invert keeps the rest.

⚙️ **NarayanaGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Narayana gate: passes note-ons on Narayana's-cows counts ($a(n)=a(n-1)+a(n-3)$: 1,2,3,4,6,9,13,19,28,...), a slow recurrence whose ratio tends to the supergolden ratio; Invert keeps the rest.

⚙️ **NarcissisticGate** [PRB][CV][MIDI] in: Audio out: MIDI

Narcissistic gate: passes note-ons on Armstrong (narcissistic) counts that equal the sum of their own digits each raised to the number-of-digits power (1,153,370,371,407,...), a rare self-referential set; Invert keeps the rest.

⚙️ **NivenGate** [PRB][CV][MIDI] in: Audio out: MIDI

Niven (Harshad) gate: passes note-ons on counts divisible by the sum of their own digits (1..10,12,18,20,21,24,...), a moderately-dense digit-driven set; Invert keeps the rest.

⚙️ **NontotientGate** [PRB][CV][MIDI] in: Audio out: MIDI

Nontotient gate: passes note-ons on nontotient counts that are never the output of Euler's totient for any number (14,26,34,38,50,...; all even), an unreachable set; Invert keeps the rest.

⚙️ **NudeNumberGate** [PRB][CV][MIDI] in: Audio out: MIDI

Nude-number gate: passes note-ons on nude counts that are divisible by each of their own nonzero digits (1..9,11,12,15,22,24,...), a polydivisible-by-digit set; Invert keeps the rest.

⚙️ **OctahedralGate** [PRB][CV][MIDI] in: Audio out: MIDI

Octahedral gate: passes note-ons on octahedral counts ($k(2k^2+1)/3$: 1,6,19,44,85,...), the 3-D figurate numbers counting points in an octahedron; Invert keeps the rest.

⚙️ **OddDigitGate** [PRB][CV][MIDI] in: Audio out: MIDI

Odd-digit gate: passes note-ons whose count is written with only odd decimal digits (1,3,5,7,9,11,13,15,...), the digit-pattern complement of the even-digit gate; Invert passes counts containing an even digit.

⚙️ **OdiousGate** [PRB][CV][MIDI] in: Audio out: MIDI

Odious gate: passes note-ons on odious counts with an odd number of 1-bits in binary (1,2,4,7,8,11,13,14,...), the Thue-Morse complement of the evil numbers; Invert keeps the evil counts.

⚙️ **OreNumberGate** [PRB][CV][MIDI] in: Audio out: MIDI

Ore-number gate: passes note-ons on Ore (harmonic-divisor) counts whose divisors have an integer harmonic mean (1,6,28,140,496,...), a divisor-rich set overlapping the perfect numbers; Invert keeps the rest.

⚙️ **PadovanGate** [PRB][CV][MIDI] in: Audio out: MIDI

Padovan gate: passes note-ons on Padovan-number counts (1,2,3,4,5,7,9,12,16,...; $P(n)=P(n-2)+P(n-3)$), the plastic-number sequence; Invert keeps the rest.

⚙️ **PalindromeGate** [PRB][CV][MIDI] in: Audio out: MIDI

Palindrome gate: passes note-ons only on counts that read the same forwards and backwards (1,2,...,9,11,22,...,101,...), an irregular, self-mirroring thinning; Invert keeps the non-palindromes.

⚙️ **PalindromicPrimeGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Palindromic-prime gate: passes note-ons on counts that are both prime and a decimal palindrome (2,3,5,7,11,101,131,151,...), the intersection of two famous sparse sets; Invert keeps the rest.

⚙️ **PalindromicSquareGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Palindromic-square gate: passes note-ons whose square reads the same forwards and backwards (1,2,3,11,22,26,101,111,...), the roots of palindromic squares; Invert keeps the rest.

⚙️ **PandigitalGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Pandigital gate: passes note-ons on 1-to-k pandigital counts whose digits are exactly a permutation of 1..(digit count), using each once with no zeros (1,12,21,123,132,...); Invert keeps the rest.

⚙️ **PartitionGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Partition gate: passes note-ons on integer-partition counts $p(k)$ (1,2,3,5,7,11,15,22,30,...), the number of ways to write k as a sum, which thins out as it climbs; Invert keeps the others.

⚙️ **PartitionParityGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Partition-parity gate: passes note-ons when the integer-partition number $p(\text{count})$ is odd, a famously-irregular parity pattern (the subject of Ramanujan's congruences); Invert keeps the even- p counts.

⚙️ **PellGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Pell gate: passes note-ons on Pell-number counts (1,2,5,12,29,70,...; $P(n)=2P(n-1)+P(n-2)$), the silver-ratio sequence behind the best rational approximations to root-2; Invert keeps the others.

⚙️ **PellLucasGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Pell-Lucas gate: passes note-ons on companion-Pell counts ($Q(n)=2Q(n-1)+Q(n-2)$: 2,6,14,34,82,198,...), the Lucas-style partner of the Pell numbers; Invert keeps the rest.

⚙️ **PentagonalGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Pentagonal gate: passes note-ons on pentagonal-number counts (1,5,12,22,35,...; $k(3k-1)/2$), the figurate numbers from Euler's pentagonal-number theorem; Invert keeps the rest.

⚙️ **PentanacciGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Pentanacci gate: passes note-ons on pentanacci counts where each term is the sum of the previous five (1,2,4,8,16,31,61,120,...), a fast-growing Fibonacci generalization; Invert keeps the rest.

⚙️ **PerfectNumberGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Perfect-number gate: passes note-ons on perfect counts equal to the sum of their proper divisors (6,28,496,8128), an exceedingly rare and ancient set; mostly silent, firing only on those landmarks. Invert keeps the rest.

 **PerfectPowerGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Perfect-power gate: passes note-ons on counts that are an exact power a^b with $b \geq 2$ (1,4,8,9,16,25,27,32,...), merging the squares, cubes and higher powers into one set; Invert keeps the rest.

 **PerrinGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Perrin gate: passes note-ons on Perrin-sequence counts (3,2,5,5,7,10,12,...; $P(n)=P(n-2)+P(n-3)$), famous because n almost always divides $P(n)$ exactly when n is prime; Invert keeps the rest.

 **PoliteGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Polite gate: passes note-ons on polite counts that can be written as a sum of two or more consecutive integers (everything except the powers of two); Invert keeps just the powers of two.

 **PolydivisibleGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Polydivisible gate: passes note-ons on polydivisible counts where the first k digits form a number divisible by k for every prefix (12,15,24,...,120,...); Invert keeps the rest.

 **PolygonalGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Polygonal gate: passes note-ons on s -gonal figurate-number counts for any chosen number of Sides - triangular at 3, square at 4, pentagonal at 5, and so on - one knob covering the whole figurate family; Invert keeps the rest.

 **PowerfulNumberGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Powerful-number gate: passes note-ons on powerful counts where every prime factor occurs at least squared (1,4,8,9,16,25,27,32,36,...), the dense-factor opposite of squarefree; Invert keeps the rest.

 **PowerOfTwoGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Power-of-two gate: passes note-ons only on counts that are exact powers of two (1,2,4,8,16,32,...), an octave-spaced exponentially-thinning pattern; Invert keeps the rest.

 **PracticalNumberGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Practical-number gate: passes note-ons on practical counts, where every smaller integer is a sum of distinct divisors of the count (1,2,4,6,8,12,...; all powers of two and more), a divisor-rich set close to the abundant numbers; Invert keeps the rest.

 **PrimeDigitGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Prime-digit gate: passes note-ons whose count is written with only the prime digits 2, 3, 5 and 7 (2,3,5,7,22,23,25,...), a digit-pattern sieve; Invert passes counts containing a non-prime digit.

🔲🔲 **PrimeGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Number-theory gate: counts incoming notes and lets one through only when its ordinal is a prime number (2,3,5,7,11,...), so the texture thins out unpredictably as primes spread; Invert keeps the composites instead.

🔲🔲 **PrimeIntervalGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Prime-interval gate: passes a note only when its leap from the previous one spans a prime number of semitones (2,3,5,7,11,...), favouring the seconds, thirds, fourths and fifths while blocking octaves and tritones - a number-theoretic melodic filter.

🔲🔲 **PrimePowerGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Prime-power gate: passes note-ons only when the count is a power of a single prime (2,3,4,5,7,8,9,...), a sparse self-similar pattern from analytic number theory; Invert keeps everything else.

🔲🔲 **PrimeTripletGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Prime-triplet gate: passes note-ons on counts belonging to a prime triplet - a prime with two more primes within a span of six (5,7,11,13,17,19,...); Invert keeps the rest.

🔲🔲 **PrimorialGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Primorial gate: passes note-ons on primorial counts - products of the first k primes (2,6,30,210,2310,...) - an extremely sparse, fast-growing set; Invert keeps the rest.

🔲🔲 **ProbabilityByPitch** [PRB] [CV] [MIDI] in: Audio out: MIDI

Probability by pitch: the chance a note passes rises or falls with its keyboard position, so you can thin out the bass while keeping the top (or the reverse) - register-weighted probabilistic gating.

🔲🔲 **ProbabilityRamp** [PRB] [CV] [MIDI] in: Audio out: MIDI

Probability ramp: the pass chance sweeps from Start to End across a repeating cycle of notes, so phrases can fade in (sparse to dense) or thin out (dense to sparse) on a loop - an evolving probabilistic gate.

🔲🔲 **PronicGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Pronic gate: passes note-ons on pronic (oblong) counts $n(n+1)$: 2,6,12,20,30,..., the products of consecutive integers; Invert keeps the rest.

🔲🔲 **ProthGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Proth gate: passes note-ons on Proth-number counts ($k \cdot 2^n + 1$ with k odd and k below 2^n : 3,5,9,13,17,25,...), the numbers with a fast dedicated primality test; Invert keeps the rest.

🔲🔲 **RandomGateWalk** [PRB] [CV] [MIDI] in: Audio out: MIDI

Random-gate walk: the pass probability itself drifts on a bounded random walk, so the music breathes between dense and sparse passages instead of thinning uniformly like a fixed-chance gate.

⚙️ **RecamanGate** [PRB][CV][MIDI] in: Audio out: MIDI

Recaman gate: passes note-ons on counts that appear in Recaman's sequence (jump back by n if new and positive, else forward), the famously erratic jumping sequence; its darting coverage thins the stream unpredictably. Invert keeps the rest.

⚙️ **RefactorableGate** [PRB][CV][MIDI] in: Audio out: MIDI

Refactorable gate: passes note-ons on refactorable (τ) counts whose own number of divisors divides the number (1,2,8,9,12,18,24,36,...); Invert keeps the rest.

⚙️ **RepdigitGate** [PRB][CV][MIDI] in: Audio out: MIDI

Repdigit gate: passes note-ons on repdigit counts whose decimal digits are all identical (1..9,11,22,...,99,111,...), a sparse uniform-digit pattern distinct from the all-ones repunits; Invert keeps the rest.

⚙️ **RepunitGate** [PRB][CV][MIDI] in: Audio out: MIDI

Repunit gate: passes note-ons on repunit counts whose decimal digits are all ones (1,11,111,1111,...), an extremely sparse, widely-spaced pattern; Invert keeps the rest.

⚙️ **RhombicDodecahedralGate** [PRB][CV][MIDI] in: Audio out: MIDI

Rhombic-dodecahedral gate: passes note-ons on rhombic-dodecahedral counts (1,15,65,175,369,...), the 3-D figurate numbers of that crystal shape; Invert keeps the rest.

⚙️ **RoughNumberGate** [PRB][CV][MIDI] in: Audio out: MIDI

Rough-number gate: passes note-ons whose count's smallest prime factor is at least Bound (a B-rough number with no small factors), the complement of smoothness; raising Bound makes the gate fire only on prime-like counts. Invert keeps the rest.

⚙️ **SafePrimeGate** [PRB][CV][MIDI] in: Audio out: MIDI

Safe-prime gate: passes note-ons on safe-prime counts - primes p for which $(p-1)/2$ is also prime (5,7,11,23,47,59,...), the partners of the Sophie-Germain primes used in cryptography; Invert keeps the rest.

⚙️ **SchroederGate** [PRB][CV][MIDI] in: Audio out: MIDI

Schroeder gate: passes note-ons on large-Schroeder counts (1,2,6,22,90,394,1806,...), the lattice-path numbers counting subdivisions and super-Catalan structures; Invert keeps the rest.

⚙️ **SelfNumberGate** [PRB][CV][MIDI] in: Audio out: MIDI

Self-number gate: passes note-ons on self (Colombian) counts that cannot be written as some smaller number plus that number's digit-sum (1,3,5,7,9,20,31,...), a self-referential sieve; Invert keeps the rest.

 **SemiperfectGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Semiperfect gate: passes note-ons on semiperfect (pseudoperfect) counts where some subset of the proper divisors sums to the number (6,12,18,20,24,28,...); Invert keeps the rest.

 **SemiprimeGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Semiprime gate: passes note-ons on semiprime counts - numbers that are the product of exactly two primes (4,6,9,10,14,15,21,...) - a moderately-sparse, multiplicatively-defined pattern; Invert keeps the rest.

 **SexyPrimeGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Sexy-prime gate: passes note-ons on counts in a sexy-prime pair (two primes six apart: 5,7,11,13,17,23,...), so named from the Latin sex for six; Invert keeps the rest.

 **SmithNumberGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Smith-number gate: passes note-ons on Smith counts whose digit sum equals the digit sum of their prime factorization (4,22,27,58,85,...), a quirky composite set; Invert keeps the rest.

 **SmoothNumberGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Smooth-number gate: passes note-ons whose count's largest prime factor stays at or below Bound (a B-smooth number, the easy-to-factor integers behind sieve algorithms); raising Bound opens the gate on more counts. Invert keeps the rest.

 **SophieGermainGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Sophie-Germain gate: passes note-ons on Sophie-Germain-prime counts - primes p where $2p+1$ is also prime (2,3,5,11,23,...), the primes behind safe-prime cryptography; Invert keeps the rest.

 **SphenicGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Sphenic gate: passes note-ons on sphenic counts - squarefree products of exactly three distinct primes (30,42,66,70,78,...) - a multiplicatively-defined, moderately-sparse set; Invert keeps the rest.

 **SquarefreeGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Squarefree gate: passes note-ons on squarefree counts with no repeated prime factor (1,2,3,5,6,7,10,11,13,...), about 61 percent of integers; Invert keeps the square-divisible counts.

 **SquareGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Square gate: passes note-ons only on perfect-square counts (1,4,9,16,25,...), so notes get steadily sparser as the gaps between squares widen; Invert plays everything except the squares.

 **SquarePyramidalGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Square-pyramidal gate: passes note-ons on square-pyramidal counts (1,5,14,30,55,...), the number of cannonballs in a square-based pyramid; Invert keeps the rest.

 **SquareTriangularGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Square-triangular gate: passes note-ons on the rare counts that are simultaneously a perfect square and a triangular number (1,36,1225,41616,...); Invert keeps the rest.

 **StarNumberGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Star-number gate: passes note-ons on centered star (hexagram) counts (1,13,37,73,121,...), the dots in a growing six-pointed star; Invert keeps the rest.

 **StellaOctangulaGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Stella-octangula gate: passes note-ons on stella-octangula counts ($k(2k^2-1)$): 1,14,51,124,245,...), the points of a growing star-tetrahedron; Invert keeps the rest.

 **StrobogrammaticGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Strobogrammatic gate: passes note-ons on counts that read the same when rotated 180 degrees (0,1,8,11,69,88,96,...), a visual digit-symmetry set; Invert keeps the rest.

 **SunnyNumberGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Sunny-number gate: passes note-ons on sunny counts where the number plus one is a perfect square (3,8,15,24,35,48,...), one less than each square; Invert keeps the rest.

 **SuperabundantGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Superabundant gate: passes note-ons on superabundant counts whose ratio $\sigma(n)/n$ beats every smaller number (1,2,4,6,12,24,36,48,60,120,...), the densest-divisor records; Invert keeps the rest.

 **SylvesterGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Sylvester gate: passes note-ons on Sylvester's-sequence counts (2,3,7,43,1807,...; each one more than the product of all previous), a doubly-exponential set so sparse it fires only a handful of times; Invert keeps the rest.

 **TetrahedralGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Tetrahedral gate: passes note-ons on tetrahedral-number counts $n(n+1)(n+2)/6$ (1,4,10,20,35,56,...), the 3-D figurate numbers counting stacked spheres; their cubic spacing thins fast. Invert keeps the rest.

 **TetranacciGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Tetranacci gate: passes note-ons on tetranacci counts where each term is the sum of the previous four (1,2,4,8,15,29,56,108,...); Invert keeps the rest.

⚙️ **ThabitGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Thabit gate: passes note-ons on Thabit (321-) number counts ($3 \cdot 2^{n-1}$: 2,5,11,23,47,95,191,...), the medieval numbers behind amicable-pair constructions; Invert keeps the rest.

⚙️ **ThueMorseGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Thue-Morse gate: passes notes where the cube-free Thue-Morse sequence (the bit-parity of the note count) is 0, an aperiodic-yet-structured pattern that never settles into a short loop; Invert flips which half plays.

⚙️ **TrailingZeroGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Trailing-zero gate: passes note-ons whose count has exactly the chosen number of trailing binary zeros (its 2-adic valuation), selecting one rung of the ruler sequence - odd counts at 0, doubly-even at higher; Invert keeps the rest.

⚙️ **TriangularGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Triangular gate: passes note-ons on triangular-number counts (1,3,6,10,15,...), thinning the stream on the handshake-number sequence; Invert keeps the non-triangular notes.

⚙️ **TribonacciGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Tribonacci gate: passes note-ons on Tribonacci-sequence counts (1,2,4,7,13,24,44,...; each the sum of the previous three), spreading wider than Fibonacci; Invert keeps the rest.

⚙️ **TrimorphicGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Trimorphic gate: passes note-ons on trimorphic counts whose cube ends in the number itself (1,4,5,6,9,24,25,49,...), the cube-analogue of automorphic numbers; Invert keeps the rest.

⚙️ **TruncatablePrimeGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Right-truncatable-prime gate: passes note-ons on primes that stay prime as each trailing digit is chopped off (2,3,5,7,23,29,31,37,53,...), a famously finite set; Invert keeps the rest.

⚙️ **TwinPrimeGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Twin-prime gate: passes note-ons on counts belonging to a twin-prime pair (a prime with another prime two away: 3,5,7,11,13,17,19,...), thinning onto the famously-clustered twin primes; Invert keeps the rest.

⚙️ **UlamGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Ulam gate: passes note-ons on Ulam-number counts (each the smallest integer that is a sum of two earlier terms in exactly one way: 1,2,3,4,6,8,11,...), a mysteriously near-periodic set; Invert keeps the rest.

⚙️ **UndulatingGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Undulating gate: passes note-ons on undulating counts whose digits zig-zag high-low-high (121,132,231,1212,...), a wave-like digit pattern; Invert keeps the rest.

 **UndulatingNumberGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Undulating-number gate: passes note-ons on counts of 3+ digits that undulate in an alternating ababab pattern with two different digits (121,131,212,232,...); Invert keeps the rest.

 **UntouchableGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Untouchable gate: passes note-ons on untouchable counts that can never be the sum of the proper divisors of any number (2,5,52,88,96,...), an unreachable set in the aliquot map; Invert keeps the rest.

 **VelocityFloorGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Velocity floor gate: drops note-ons whose velocity falls below a floor, filtering out the quietest notes (off events for dropped notes are removed too to avoid stuck notes); Invert keeps only the quiet ones.

 **VelocityProbGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Velocity-probability gate: passes each note with a probability that rises with its velocity, so soft notes thin out and accents survive - a dynamics-driven random thinning; Bias lifts the baseline pass chance.

 **VelRandomGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Velocity-random gate: passes each note with a probability that rises with its velocity, so accented hits survive while ghost notes thin out - dynamics-weighted probabilistic gating.

 **WedderburnEthingtonGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Wedderburn-Ethington gate: passes note-ons on the counts of unordered binary trees (1,2,3,6,11,23,46,98,...), a combinatorial sequence from chemistry and phylogenetics; Invert keeps the rest.

 **WeirdNumberGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Weird-number gate: passes note-ons on weird counts that are abundant yet have no subset of proper divisors summing to the number (70,836,4030,...), a rare paradoxical set; Invert keeps the rest.

 **WoodallGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Woodall gate: passes note-ons on Woodall-number counts ($n \cdot 2^{n-1}$: 1,7,23,63,159,383,...), an exponentially-sparse set partnered with the Cullen numbers; Invert keeps the rest.

 **WythoffGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Wythoff gate: passes note-ons on the lower-Wythoff counts $\text{floor}(n \cdot \phi)$ (1,3,4,6,8,9,11,...), the golden-ratio Beatty sequence that, with its complement, partitions the integers; Invert keeps the upper-Wythoff notes.

 **ZuckermanGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Zuckerman gate: passes note-ons on Zuckerman counts divisible by the product of their own (nonzero) decimal digits (1..9,11,12,15,24,36,...); Invert keeps the rest.

 **ZuckermanGate** [PRB] [CV] [MIDI] in: Audio out: MIDI

Zuckerman gate: passes note-ons on Zumkeller counts whose divisors can be split into two sets of equal sum (6,12,20,24,28,30,...), a balanced-divisor set; Invert keeps the rest.

Routing (29)

← **Channel** [RTE] [MIDI] in: Audio out: MIDI

Reassigns notes to a fixed MIDI channel.

← **ChannelByPitch** [RTE] [MIDI] in: Audio out: MIDI

Channel-by-pitch: routes notes to different MIDI channels by register, splitting the keyboard into Zones equal bands so a single part can drive several multitimbral instruments stacked across the range.

← **ChannelByZone** [RTE] [MIDI] in: Audio out: MIDI

Channel by zone: routes notes below the Split point to channel ChanLo and notes at or above it to ChanHi, splitting one keyboard across two MIDI channels (multitimbral split).

← **ClampToRange** [RTE] [MIDI] in: Audio out: MIDI

Clamp to range: pulls every out-of-range note onto the nearest boundary of [Low,High], folding a wide part flat against the edges of a fixed register.

← **FixedNote** [RTE] [MIDI] in: Audio out: MIDI

Forces every note to a single fixed pitch (drum / trigger).

← **FoldToRange** [RTE] [MIDI] in: Audio out: MIDI

Fold to range: shifts every note up or down by whole octaves until it sits inside the [Low,High] window, packing a wide-range part into a fixed register while preserving each note's pitch class.

← **KeyboardSplit3** [RTE] [MIDI] in: Audio out: MIDI

Three-zone split: divides the keyboard at two split points into low, mid and high zones, each transposed by its own amount - a quick way to layer a bassline, a pad and a lead from one part across three regions.

← **KeySwitch** [RTE] [MIDI] in: Audio out: MIDI

Splits the keyboard at a note; one zone passes, the other is filtered.

➤ **KeyZoneGate** [RTE] [MIDI] in: Audio out: MIDI

Key-zone gate: passes only notes whose pitch sits inside [Low,High] and drops the rest (their note-offs are removed too to avoid stuck notes); Invert keeps only the notes outside the zone.

➤ **MaxPolyphony** [RTE] [MIDI] in: Audio out: MIDI

Max polyphony: caps how many notes may sound at once, dropping any note-on past the limit until a voice frees up - a voice-stealing-free polyphony clamp for thinning dense chords.

➤ **Merge** [RTE] [MIDI] in: Audio out: MIDI

Passes both note-stream inputs through, merged into one stream.

➤ **MidiLatch** [RTE] [MIDI] in: Audio out: MIDI

Holds notes after release until the next note arrives.

➤ **MirrorZone** [RTE] [MIDI] in: Audio out: MIDI

Mirror zone: reflects the notes inside [Low,High] around the zone centre (a melodic inversion confined to one key zone), leaving notes outside the zone alone.

➤ **MuteZone** [RTE] [MIDI] in: Audio out: MIDI

Mute zone: drops every note whose pitch sits inside [Low,High] while letting the rest of the keyboard through, punching a silent notch into the range.

➤ **NoteFilter** [RTE] [MIDI] in: Audio out: MIDI

Passes only notes inside a key and velocity window.

➤ **NoteLimit** [RTE] [MIDI] in: Audio out: MIDI

Polyphony cap: never lets more than Max notes sound at once, stealing the oldest (FIFO note-off) when a new note would exceed the limit - tames dense chords and runaway generators.

➤ **NoteRangeTranspose** [RTE] [MIDI] in: Audio out: MIDI

Note-range transpose: transposes by Semitones only the notes whose pitch sits inside [Low,High], leaving everything outside the zone at its original pitch.

➤ **NoteToGateCC** [RTE] [MIDI] in: Audio out: MIDI

Note-to-gate CC: emits a chosen controller high while any note is on and low when it releases, turning note events into a gate/trigger control signal for envelopes, side-chains or external gear.

➤ **NoteToPitchBend** [RTE] [MIDI] in: Audio out: MIDI

Note-to-pitch-bend: emits a pitch-bend proportional to each note's distance from a center pitch, so notes far from the center are pre-bent - a way to map keyboard position onto continuous detune or to drive a synth's bend input from played pitch.

➤ **PanByPitch** [RTE] [MIDI] in: Audio out: MIDI

Pan by pitch: emits a pan CC (controller 10) that places low notes to the left and high notes to the right, an automatic keyboard-position stereo spread as a synth or sampler plays.

➤ **PolyChan** [RTE] [MIDI] in: Audio out: MIDI

Round-robins each new note to the next of Voices MIDI channels from Base, spreading a polyphonic part across a multitimbral rig - one note per channel, the classic voice-allocation split (distinct from MPEConvert's per-note MPE channels).

➤ **RandomPanCC** [RTE] [MIDI] in: Audio out: MIDI

Random-pan CC: fires a random pan (controller 10) value on every note-on, scattering successive notes across the stereo field for an automatic, ever-shifting spatial spread; Spread sets how wide the scatter.

➤ **Split** [RTE] [MIDI] in: Audio out: MIDI

Keyboard split: notes below the split point transpose by Low, the rest by High.

➤ **SplitTranspose** [RTE] [MIDI] in: Audio out: MIDI

Split transpose: a keyboard split where notes at or above the Split point are transposed by Semitones and notes below pass untouched, so two hands play in different registers.

➤ **Thin** [RTE] [MIDI] in: Audio out: MIDI

Note-density limiter: drops any note-on arriving within MinGap ms of the last one passed, decluttering a busy stream or fast repeats - distinct from NoteLimit, which caps simultaneous polyphony. A dropped note's off is dropped too.

➤ **Transformer** [RTE] [MIDI] in: Audio out: MIDI

Rule: notes within a range are transposed and velocity-scaled.

➤ **VelocityZoneGate** [RTE] [MIDI] in: Audio out: MIDI

Velocity-zone gate: passes only notes whose velocity falls inside [Min,Max] and drops the rest; Invert keeps only the notes outside the velocity window.

➤ **WrapAround** [RTE] [MIDI] in: Audio out: MIDI

Wrap around: notes that rise above the High edge wrap back to the Low end of the range (and vice versa), a modular pitch fold that keeps a part inside a register while letting runs cycle around.

◀ **ZoneOctaveShift** [RTE] [MIDI] in: Audio out: MIDI

Zone octave shift: shifts the notes inside [Low,High] by whole Octaves, leaving notes outside the zone where they are, a register jump confined to one key range.

Sequencing (63)

■□□ **Appoggiatura** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Leaning grace note: each struck note first sounds a long auxiliary (Interval away) that takes Time from the principal, which then sustains - the expressive long grace, unlike the quick Flam or Mordent.

■□□ **Arp** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Arpeggiator: cycles held notes through a pattern at a synced rate with gate, octaves, swing and step count. Latch holds the chord so it keeps arpeggiating after the keys are released (the next key starts a new chord).

■□□ **BeattyGate** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Beatty gate: passes note-ons on a Beatty sequence floor($n \cdot \text{Ratio}$) for any irrational ratio, a Sturmian, maximally-even thinning whose density is set by the ratio; Invert keeps the complement.

■□□ **BurstGate** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Burst gate: lets OnLen consecutive notes through, mutes the next OffLen, and repeats, chopping a steady run into rhythmic bursts and rests - a note-count gate for stutter and ratchet feels.

■□□ **Cascade** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Each struck note fans out into a timed run of Steps notes climbing (or falling) the scale from it - an arpeggiated flourish fired per key, unlike Arp which cycles the whole held chord.

■□□ **ChordArp** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Cthulhu-style chord arpeggiator: each key builds a full chord (Type) and the result is arpeggiated upward across Octaves at a synced Rate - one finger plays a moving arpeggio of the whole chord. Releasing the key stops its run.

■□□ **ClaveGate** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Clave gate: passes notes on the pulses of the Afro-Cuban son clave, the five-stroke rhythmic key of salsa and Latin music, selectable as the 3-2 or 2-3 orientation; Invert keeps the off-clave notes.

■□□ **ClockDivideGate** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Clock divider: passes one note-on out of every Divide that arrive and drops the rest, a simple note-rate divider for thinning fast passages or extracting a downbeat pulse.

■□□ **CongruentGate** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Congruent gate: passes a note only when its running count is congruent to Remainder modulo Modulus, a fully-parameterized periodic gate that covers every-Nth-note and offbeat patterns from two numbers; Invert keeps the rest.

■□□ **CrossPulseGate** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Cross-pulse gate: lays two pulse grids of different periods over the note stream and passes notes where they coincide (AND) or where either fires (OR), generating polyrhythmic and cross-rhythmic gating from two simple counts.

■□□ **Drunk** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Random-walk melodic sequencer: each synced Rate step nudges the pitch by a random +/-Step semitones, bounded within +/-Range of the lowest held note - Brownian melody anchored to the chord, unlike MidiRandom's memoryless picks.

■□□ **DurationFromPitch** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Duration from pitch: maps each note's pitch to its length so high notes play short and low notes long (or, inverted, the reverse), a register-tilt articulation.

■□□ **Echo** [SEQ] [CV] [MIDI] in: Audio out: MIDI

MIDI delay: repeats notes at a synced time with velocity feedback decay.

■□□ **EuclidComplementGate** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Euclidean complement: passes notes that fall on the rests of a Euclidean rhythm of Pulses-in-Steps, the photo-negative of a Euclid sequencer - an instant interlocking counter-rhythm.

■□□ **EuclideanGate** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Euclidean gate: passes notes on the pulses of a Euclidean (Bjorklund) rhythm that spreads Pulses as evenly as possible over Steps - the algorithm that reproduces clave, tresillo and most world rhythms from two numbers; Rotation shifts the pattern. Invert keeps the rest.

■□□ **EuclidMel** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Euclidean melody: spreads Pulses evenly across Steps (Bjorklund) and, on each hit, advances a cursor through the held chord - the rhythm is Euclidean and the pitch walks the chord, unlike SeqEuclid which stabs all held notes per pulse.

■□□ **Flam** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Precedes every note with a quiet grace note, delaying the main hit so the grace leads it - the classic drum flam.

■□□ **GateArp** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Cthulhu's arp gate lane: walks the held chord upward across Octaves while the note length cycles a per-step staccato/legato pattern, so the rhythm breathes - short stabs then sustained notes. Gate scales the pattern (>1 overlaps into the next step).

■□■ **GateLengthAccent** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Gate-length accent: holds every Nth note (the downbeat) for the Long duration and clips the rest to Short, a rhythmic gate accent.

■□■ **GateLengthAlternate** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Gate-length alternate: alternates note durations between two lengths (long, short, long, short), a bouncing gate groove.

■□■ **GateLengthFromInterval** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Gate-length from interval: grows each note's duration with how far it leaps from the previous note, so big melodic jumps ring longer than stepwise motion.

■□■ **GateLengthInvVelocity** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Gate-length inverse velocity: louder notes are clipped short and soft notes ring long (the inverse of Vellength), a punch-then-decay articulation.

■□■ **GateLengthLFO** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Gate-length LFO: sweeps note duration between Min and Max with a sine that completes one cycle every Period notes, a breathing gate that swells and shrinks.

■□■ **GateLengthRamp** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Gate-length ramp: ramps note duration linearly from Min to Max across Steps notes then resets, an automatic gate crescendo.

■□■ **GateLengthRandom** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Gate-length random: gives each note a random duration between Min and Max, humanising the gate so no two notes ring exactly the same length.

■□■ **GoldenWordGate** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Golden-word gate: gates notes by the infinite Fibonacci word (the Sturmian 0/1 sequence at golden-ratio spacing), the most-ordered aperiodic rhythm there is; Invert flips which letter plays.

■□■ **KolakoskiGate** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Kolakoski gate: gates notes by the self-describing Kolakoski 1,2 sequence (whose run-lengths reproduce the sequence itself), a hypnotically self-similar aperiodic pattern; Invert flips which symbol plays.

■□■ **MidiBrianBrain** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Brian's Brain MIDI sequencer: an 8x8 three-state grid (ready / firing / dying) evolves under Brian's Brain rules - a ready cell ignites only with exactly two firing neighbours, firing cells pass to dying, dying cells reset. Each clock step plays a note for every firing cell in the next column (row maps to a scale degree from Root/Scale); a full 8-step

sweep advances one generation. The refractory state keeps it restless and sparkly where Game of Life settles. Density sets the random fill. The MIDI-note counterpart to the BrianBrain CV block.

■□■ **MidiGameOfLife** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Conway's Game of Life MIDI sequencer: an 8x8 cell grid evolves under the B3/S23 rule; each clock step reads the next column and plays a note for every live cell (row maps to a scale degree from Root/Scale), and a full 8-step sweep advances one generation. Gliders, blinkers and still-lives become ever-shifting melodic and harmonic patterns. Density sets the random fill when the grid (re)seeds. The MIDI-note counterpart to the GameOfLife CV block.

■□■ **MidiLangton** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Langton's Ant MIDI melody: a single turmite walks an 8x8 grid - turning right on a white cell, left on a black one, flipping each cell as it leaves, then stepping forward. Each clock advances the ant Steps cells and plays one note for its current row (mapped to a scale degree from Root/Scale), so the melody scribbles chaotically then locks into the famous periodic 'highway'. Emergent order from one trivial rule - a single moving agent, not a population.

■□■ **MidiMarkov** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Learns the note-to-note transition statistics of what you play (a 12x12 pitch-class chain) and, on each synced step, walks the chain to improvise a new line in the same style. Memory fades old transitions.

■□■ **MidiQuantize** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Snaps note starts to a synced grid by a strength amount.

■□■ **MidiTranceGate** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Chops the held notes with a 16-step on/off Pattern at a synced Rate: every 'on' step retriggers all held notes, gated to a fraction of the step. The rhythmic gate behind the trance sound - pattern-driven, where Roll is a fixed pulse.

■□■ **MidiTuring** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Turing-machine sequencer (Music Thing style): a circular shift register clocks once per step and the held notes fire when the read bit is 1. Change is the probability the recycled bit flips - 0 locks an evolving loop, 1 fully randomises, between = slowly mutating melodies.

■□■ **Mordent** [SEQ] [CV] [MIDI] in: Audio out: MIDI

One-shot ornament: each struck note plays principal -> auxiliary -> principal at the attack, then sustains, the single-flick counterpart to the continuous Trill.

■□■ **NoteLength** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Forces a fixed note duration (gate length).

■□■ **NoteOffDelay** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Extends each note's gate by delaying its note-off.

■□■ **OctaveArp** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Cthulhu's arp octave lane: arpeggiates the held chord while a per-step octave-offset Pattern leaps the line up and down by whole octaves (alternating, staircase, bounce...) instead of climbing monotonically like StepArp.

■□■ **PaperfoldGate** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Paperfold gate: gates notes by the regular paperfolding (dragon-curve) sequence of the count, a 2-automatic pattern that folds in on itself at every scale; Invert flips the kept and dropped halves.

■□■ **PatternGate** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Gates note-ons through a 16-step on/off pattern (bitmask).

■□■ **Polymeter** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Two step lanes of different lengths (LenA, LenB) clock together over the held notes, phasing against each other so the combined pattern only repeats after lcm(LenA, LenB) steps - instant polymetric movement from one chord.

■□■ **PolyrhythmGate** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Polyrhythm gate: passes notes that land on a multiple of either count A or count B, overlaying two pulse grids (e.g. 3 against 4) into an interlocking cross-rhythm of accents.

■□■ **RatchetArp** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Cthulhu's arp repeat lane: walks the held chord upward across Octaves while a per-step pattern retriggers selected steps multiple times (capped by Max), packing rolls into the line. Unlike SeqRatchet's fixed count, the repeat count varies per step.

■□■ **Ricochet** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Accelerating bouncing retriggers with decaying velocity.

■□■ **Roll** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Drum roll / buzz: retriggers every held note at Rate for as long as it is held, each hit gated to a fraction of the interval.

■□■ **SeqEuclid** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Euclidean rhythm gate: spreads Pulses evenly across Steps (rotatable) at a synced rate.

■□■ **SeqRatchet** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Retriggers each note Count times within a synced step.

■□□ **Shift**[SEQ][CV][MIDI] in: Audio out: MIDI

Delays every note by Delay ms (groove nudge / lay-back), pushing note-ons and offs back by the same amount so durations are preserved - the per-track timing offset of a groove sequencer.

■□□ **SkipArp**[SEQ][CV][MIDI] in: Audio out: MIDI

Cthulhu's arp skip lane: arpeggiates the held chord upward across Octaves, but each step has a Skip probability of being dropped to a rest, thinning the pattern differently every cycle for generative movement.

■□□ **SkipNote**[SEQ][CV][MIDI] in: Audio out: MIDI

Skip note: drops every Nth note-on and lets the rest through, the inverse of a clock divider - a quick way to punch regular holes in a run of notes.

■□□ **Spiral**[SEQ][CV][MIDI] in: Audio out: MIDI

Transposing MIDI delay: each repeat is delayed by Time and shifted by Interval semitones with velocity decaying by Feedback, so one note spins off a climbing or falling staircase of echoes - unlike Echo, which repeats the same pitch.

■□□ **StaccatoCut**[SEQ][CV][MIDI] in: Audio out: MIDI

Staccato cut: clips every note to a short fixed length for a crisp detached articulation, no matter how long the key was held.

■□□ **StepArp**[SEQ][CV][MIDI] in: Audio out: MIDI

Cthulhu-style step arpeggiator: walks the held chord strictly upward across Octaves octaves at a synced Rate, accenting the first step of each cycle - the rolling, octave-jumping arp of Xfer Cthulhu's sequencer. Distinct from Arp's mode-cycling: this climbs the chord in order.

■□□ **StepSeqMidi**[SEQ][CV][MIDI] in: Audio out: MIDI

Melodic step sequencer - semitone offsets (node config) clock the held root.

■□□ **Strum**[SEQ][CV][MIDI] in: Audio out: MIDI

Spreads a chord's notes over time, up or down, like a guitar strum.

■□□ **Swing**[SEQ][CV][MIDI] in: Audio out: MIDI

Pushes off-beat eighth-notes later for a swing/shuffle feel.

■□□ **TenutoHold**[SEQ][CV][MIDI] in: Audio out: MIDI

Tenuto hold: extends every note to a long fixed length so notes sustain and run into each other, a held legato-style gate.

■ ■ ■ **TransArp** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Cthulhu's arp transpose lane: arpeggiates the held chord upward across Octaves while a per-step semitone Pattern (fifth pedals, triad arps, scalar runs) is added on top, so the line outlines a melodic contour, not just the chord tones.

■ ■ ■ **TresilloGate** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Tresillo gate: passes notes on the tresillo 3-3-2 pattern, the eight-step rhythmic cell underlying reggaeton, habanera and countless Latin grooves; Invert keeps the rest.

■ ■ ■ **Trill** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Keyboard trill: while a note is held, alternates rapidly between it and the note Interval semitones above, at Rate.

■ ■ ■ **Tuplet** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Snap note starts to a Tuples-per-beat grid (3 = triplets, 5 = quintuplets) by Strength, for a triplet or odd-tuplet feel where MidiQuantize only snaps to the straight grid.

■ ■ ■ **Turn** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Gruppetto turn: a four-step ornament at the attack - upper auxiliary, principal, lower auxiliary, then the principal held. Time sets each step, Interval the neighbour distance.

■ ■ ■ **VelArp** [SEQ] [CV] [MIDI] in: Audio out: MIDI

Cthulhu's arp velocity lane as a ramp: arpeggiates the held chord while velocity sweeps from From to To across Steps then resets, so every pass swells or fades without a per-step editor.

Timing (2)

☐ **HumanizeTiming** [FX] [MIDI] in: Audio out: MIDI

Humanize timing: nudges each event's timing by a small random amount (up to the set sample range) so a rigid sequenced part feels played by hand.

☐ **StrumChord** [FX] [MIDI] in: Audio out: MIDI

Strum chord: staggers the start of simultaneously-played notes by a small delay so a chord is strummed across the strings rather than struck as a block, with selectable up- or down-stroke direction; Spread sets the strum speed.

Tuning (3)

U **JustTune** [TUN] [MIDI] in: Audio out: MIDI

Adaptive just intonation: retunes each note toward its 5-limit JI ratio relative to Key via per-note detune (polyphonic), Amount blending from equal temperament to pure - the beatless thirds and fifths of real harmony.

U **Microtuner** [TUN] [MIDI] in: Audio out: MIDI

Stretched-octave microtuning via per-note pitch detune.

U **TuneTable** [TUN] [MIDI] in: Audio out: MIDI

Custom microtuning: detunes each note by a per-pitch-class cents offset from the node's 12-value config list (relative to Key) - meantone, Pythagorean, maqam or gamelan, any temperament. Amount scales the table.

Voicing (5)

⊗ **MidiUnison** [VOI] [MIDI] in: Audio out: MIDI

Doubles each note into multiple voices, optionally spread across channels.

⊗ **Mono** [VOI] [MIDI] in: Audio out: MIDI

Monophonic note priority (last / lowest / highest).

⊗ **MPEConvert** [VOI] [MIDI] in: Audio out: MIDI

Assigns each note its own rotating channel for per-note MPE expression.

⊗ **Slide** [VOI] [MIDI] in: Audio out: MIDI

Cthulhu's slide: mono legato glue. Picks one held note by Priority (last/low/high) and emits the new note-on before releasing the old one, so a portamento synth slurs between notes instead of retriggering.

⊗ **Sustain** [VOI] [MIDI] in: Audio out: MIDI

Sustain-pedal simulation: while Hold is engaged, note-offs are captured so the notes keep sounding; when Hold drops, every held note releases at once. Hold is a parameter, so a CC, an LFO or automation can play the pedal.

Index (563 blocks, alphabetical)

AbundanceVelocity	6	ChannelByPitch	53	DivisorProductVelocity	10
AbundancyVelocity	6	ChannelByZone	53	DivisorSumVelocity	10
AbundantGate	34	Chord	23	DoubleFactorialGate	39
Accent	6	ChordArp	56	Drone	24
AccentEveryN	6	ChordLock	23	Drunk	57
AccentOnHigh	7	ChordMem	23	DudneyGate	39
AccentOnLow	7	ChordProg	23	DurationFromPitch	57
AccentPattern	7	ChordSpanCC	29	Echo	57
AchillesGate	34	ChordVoicing	23	EmirpGate	39
AdditivePrimeGate	34	CircularPrimeGate	36	EquidigitalGate	39
AliquotStepsVelocity	7	ClampToRange	53	EuclidComplementGate	57
AliquotVelocity	7	ClaveGate	56	EuclideanAccentVelocity	10
AlmostPrimeGate	34	ClockDivideGate	56	EuclideanGate	57
AlternatingSumVelocity	7	CollatzGate	37	EuclideanVelocity	10
AntiprimeGate	34	CollatzOddStepsVelocity	8	EuclidMel	57
AntiRepeatGate	34	CollatzPeakGate	37	EvenDigitGate	39
Appoggiatura	56	CollatzPeakVelocity	8	EvenDigitSumGate	39
Arp	56	CollatzVelocity	8	ExpDecayVelocity	10
AscendingDigitGate	35	CompositeRunVelocity	8	ExpressionSwell	29
ATProc	6	CongruentGate	56	ExtravagantGate	39
AutoInvert	23	ConsonanceGate	24	FactorialBaseVelocity	10
AutomorphicGate	35	ContourGate	24	FactorialModVelocity	10
AvgPitchCC	28	ContourReverseGate	24	Fall	23
BackbeatAccent	7	ContraryGate	37	FermatPrimeGate	40
Bassline	23	CoprimeGate	37	FibGate	40
BeattyGate	56	CoprimeStepGate	37	FibonacciGate	40
BeattyVelocity	7	CototientVelocity	8	FibonacciPrimeGate	40
BellNumberGate	35	Counterpoint	24	FibVelocity	10
BendFromVelocity	28	CousinPrimeGate	37	FixedNote	53
BendProc	6	CrescendoLoop	8	Flam	57
BernoulliSignVelocity	7	CrescendoRamp	8	FoldToRange	53
BinaryPalindromeGate	35	CrossPulseGate	57	FrugalGate	40
BinaryRunVelocity	7	CubeGate	37	GamelanDetune	29
BinaryWeightGate	35	CullenGate	37	GapGate	40
BinaryWeightVelocity	8	DeficientGate	37	GateArp	57
BluesInflect	28	DelannoyGate	37	GateLengthAccent	57
BlumIntegerGate	35	DescendingDigitGate	38	GateLengthAlternate	58
BohlenPierceTune	28	Diatonic	24	GateLengthFromInterval	58
BurstGate	56	DiatonicChord	24	GateLengthInvVelocity	58
ByzantineTune	28	DiatonicHarmonize	24	GateLengthLFO	58
CakeNumberGate	35	DiatonicTranspose	24	GateLengthRamp	58
CalkinWilfVelocity	8	DigitalRootGate	38	GateLengthRandom	58
CarlosAlphaTune	28	DigitalRootPrimeGate	38	GcdVelocity	11
CarlosBetaTune	29	DigitalRootVelocity	9	GeneralizedPentagonalGate	40
CarlosGammaTune	29	DigitCountGate	38	Generative	29
CarmichaelGate	35	DigitEntropyVelocity	9	GhostNote	24
Cascade	56	DigitFactorialGate	38	GhostNoteEveryN	11
CatalanGate	35	DigitGate	38	GhostNoteInject	11
CatalanParityGate	36	DigitMaxVelocity	9	Glissando	25
CatalanVelocity	8	DigitMinVelocity	9	GoldbachCountVelocity	11
CC2Note	6	DigitProductGate	38	GoldenAngleDetune	29
CCGlide	29	DigitProductPrimeGate	38	GoldenAngleVelocity	11
CCLFO	29	DigitProductVelocity	9	GoldenGate	40
CCMap	6	DigitReverseVelocity	9	GoldenSectionAccent	11
CCSeq	29	DigitRotateVelocity	9	GoldenVel	11
CenteredCubeGate	36	DigitSpreadVelocity	9	GoldenWordGate	58
CenteredDecagonalGate	36	DigitSumGate	38	GolombGate	40
CenteredHexGate	36	DigitSumSquareGate	38	GravityQuantize	25
CenteredNonagonalGate	36	DigitSumVelocity	9	GrayCodeVelocity	11
CenteredPentagonalGate	36	DiminuendoRamp	9	HammingDistanceVelocity	11
CenteredSquareGate	36	DistinctPrimeVelocity	10	HammingNumberGate	40
CenteredTriangularGate	36	DivisorCountGate	39	HappyGate	41
Chance	36	DivisorCountVelocity	10	HappyVelocity	11
Channel	53	DivisorParityGate	39	HarmonicNumberVelocity	12

HarmonicQuantize	25	MidiTuning	59	PitchBendLFO	32
HarmonicSeriesSnap	30	MidiUnison	63	PitchClassCC	32
HarmonicSeriesTune	30	MidiVibrato	23	PitchClassGate	26
HarmonicVelocity	12	MirrorPivot	25	PitchClassVelocity	14
Harmonize	25	MirrorZone	54	PitchDrift	32
HeptanacciGate	41	MobiusVelocity	13	PitchMirror	26
HexagonalPyramidalGate	41	ModalShift	26	PitchScoop	32
HexanacciGate	41	ModInverseVelocity	13	PoliteGate	46
HighlyAbundantGate	41	ModRangeVelocity	13	PolyChan	55
HighlyCompositeGate	41	MoebiusGate	43	PolyCountCC	32
HighNoteCC	30	MoebiusVelocity	13	PolydivisibleGate	46
HirajoshiTune	30	Mono	63	PolygonalGate	46
HoaxNumberGate	41	MoranGate	43	Polymeter	60
HofstadterQGate	41	Mordent	59	PolyrhythmGate	60
Humanize	12	MotzkinGate	43	PopcountVelocity	14
HumanizeTiming	62	MPEConvert	63	PowerChordAdd	27
HumanizeVelocity	12	MultiplicativePersistenceGate	43	PowerfulNumberGate	46
IntervalCC	30	MuteZone	54	PowerOfTwoFactorVelocity	15
IntervalClassGate	25	NarayanaGate	43	PowerOfTwoGate	46
IntervalGate	41	NarayanaVelocity	14	PracticalNumberGate	46
IntervalVelocity	12	NarcissisticGate	43	PressSwell	32
Invert	25	NEdoRetune	31	PrimeCountVelocity	15
JacobiVelocity	12	NegativeHarmony	26	PrimeDigitGate	46
JacobsthalGate	41	NeoRiemann	26	PrimeFactorVelocity	15
JacobsthalVelocity	12	NibbleSwapVelocity	14	PrimeGapVelocity	15
JustMinorTune	30	NivenGate	44	PrimeGate	47
JustTune	62	NontotientGate	44	PrimeIndexAccent	15
KaprekarGate	42	Note2CC	6	PrimeIndexVelocity	15
KaprekarVelocity	12	NoteFilter	54	PrimeIntervalGate	47
KeithNumberGate	42	NoteLength	59	PrimePiVelocity	15
KeyboardSplit3	53	NoteLimit	54	PrimePowerGate	47
KeySwitch	53	NoteOffDelay	59	PrimeTripletGate	47
KeyZoneGate	53	NoteOnTrigCC	31	PrimeVelocity	15
KolakoskiGate	58	NoteRangeTranspose	54	PrimorialGate	47
KolakoskiVelocity	12	NoteToGateCC	54	PrimorialModVelocity	15
LazyCatererGate	42	NoteToPitchBend	54	ProbabilityByPitch	47
LeapGate	25	NoteToPressure	31	ProbabilityRamp	47
LeonardoGate	42	NudeNumberGate	44	PronicGate	47
LeonardoVelocity	12	OctahedralGate	44	ProthGate	47
LeylandGate	42	Octave	26	PythagoreanTune	32
LiouvilleGate	42	OctaveArp	60	QuarterToneShift	32
LogVelocity	13	OctaveClamp	26	QuarterToneTune	32
LookSayVelocity	13	OctaveFoldGate	26	RadicalVelocity	15
LowNoteCC	30	OctaveStackAdd	26	RagaBhairavTune	33
LucasGate	42	OddDigitGate	44	Ramp	16
LucasPrimeGate	42	OdiousGate	44	RandOctave	33
LucasVelocity	13	OmegaVelocity	14	RandomCC	33
LuckyNumberGate	42	OreNumberGate	44	RandomGateWalk	47
MagicConstantGate	43	OvertoneStack	26	RandomHoldVelocity	16
MaqamRastTune	30	PadovanGate	44	RandomPanCC	55
MarkovVelocity	13	PadovanVelocity	14	RangeCompress	27
MaxPolyphony	54	PalindromeGate	44	RangeExpand	27
MeantoneTune	30	PalindromicPrimeGate	45	RangeGate	27
MelodicAccent	13	PalindromicSquareGate	45	RatchetArp	60
Merge	54	PanByPitch	55	RecamanGate	48
MersenneGate	43	PandigitalGate	45	RefactorableGate	48
MersennePrimeGate	43	PaperfoldGate	60	ReflectOctave	27
MertensGate	43	PartchTune	31	RegisterAccent	16
Microtuner	62	PartitionGate	45	RepdigitGate	48
MicrotuneSpread	30	PartitionParityGate	45	RepeatDecay	16
MidiBrianBrain	58	PartitionVelocity	14	RepeatedNoteGate	27
MidiCluster	25	PatternGate	60	RepunitGate	48
MidiEnv	31	PellGate	45	ReverseBitsVelocity	16
MidiFold	25	PellLucasGate	45	ReverseSubtractVelocity	16
MidiGameOfLife	59	PellVelocity	14	RhombicDodecahedralGate	48
MidiLangton	59	PelogTune	31	Ricochet	60
MidiLatch	54	PentagonalGate	45	Roll	60
MidiLFO	31	PentanacciGate	45	RoughnessVelocity	16
MidiMarkov	59	PerfectNumberGate	45	RoughNumberGate	48
MidiQuantize	59	PerfectPowerGate	46	RudinShapiroAccent	16
MidiRandom	31	PerrinGate	46	RulerVelocity	16
MidiSampleHold	31	PerrinVelocity	14	RunningSumVelocity	16
MidiSlew	31	PersianTune	32	SafePrimeGate	48
MidiTranceGate	59	PersistenceVelocity	14	Scale	27

ScaleDegreeAccent	17	Sustain	63	VelGate	19
ScaleHighlight	17	SwellPressure	33	VelGateHysteresis	19
SchroederGate	48	Swing	61	VelInvert	20
Scoop	23	SwingVelocity	18	VelKey	20
SelfNumberGate	48	SylvesterGate	50	VelLength	20
SemiperfectGate	49	TenutoHold	61	Velocity	20
SemiprimeGate	49	TernaryDigitVelocity	18	VelocityAlternate	20
SeptimalTune	33	TetrahedralGate	50	VelocityBoost	20
SeqEuclid	60	TetranacciGate	50	VelocityClip	20
SeqRatchet	60	ThabitGate	50	VelocityCompress	20
SexyPrimeGate	49	ThaiTune	33	VelocityCurve	20
Shift	60	Thin	55	VelocityDrift	21
SigmaVelocity	17	ThueMorseGate	51	VelocityExpand	21
SineWaveVelocity	17	ThueMorseTernaryVel	18	VelocityFloor	21
SkipArp	61	ThueMorseVelocity	18	VelocityFloorGate	52
SkipNote	61	TotientRatioVelocity	18	VelocityFoid	21
SlendroTune	33	TotientStepsVelocity	18	VelocityFromInterval	21
Slide	63	TotientVelocity	18	VelocityFromPitch	21
SmithNumberGate	49	TrailingOnesVelocity	18	VelocityGammaCurve	21
SmoothnessVelocity	17	TrailingZeroGate	51	VelocityGate	21
SmoothNumberGate	49	TransArp	61	VelocityLFO	21
SophieGermainGate	49	Transformer	55	VelocityMultiply	21
SphenicGate	49	Transpose	27	VelocityProbGate	52
Spiral	61	TresilloGate	62	VelocityQuantize	22
Split	55	TriangularGate	51	VelocityRandomScale	22
SplitTranspose	55	TriangularModVelocity	19	VelocityRandomWalk	22
SquareDistanceVelocity	17	TribonacciGate	51	VelocitySmooth	22
SquareFreeGate	49	TribonacciVelocity	19	VelocitySwap	22
SquareGate	49	Trill	62	VelocityZoneGate	55
SquarePyramidalGate	49	TrimorphicGate	51	VelRandomGate	52
SquareRootVelocity	17	TruncatablePrimeGate	51	VelStep	22
SquareTriangularGate	50	TuneTable	63	VelToCC	34
SquareWaveVelocity	17	Tuplet	62	VelToDetune	34
StaccatoCut	61	TurkishMakamTune	33	VoiceLead	28
StaircaseVelocity	17	Turn	62	WedderburnEthingtonGate	52
StarNumberGate	50	TurnaroundAccent	19	WeirdNumberGate	52
StellaOctangulaGate	50	TwelveToneRow	27	WerckmeisterTune	34
StepArp	61	TwinPrimeGate	51	WholeToneSnap	28
StepSeqMidi	61	UlamGate	51	WoodallGate	52
SternBrocotVelocity	18	UndulatingGate	51	WrapAround	55
SternDiatonicVelocity	18	UndulatingNumberGate	52	WythoffGate	52
StretchTuning	33	UnisonDetuneAdd	28	WythoffVelocity	22
StrobogrammaticGate	50	UnitaryDivisorVelocity	19	YoungTune	34
Strum	61	UntouchableGate	52	ZeckendorfVelocity	22
StrumChord	62	VelArp	62	ZipfVelocity	22
SubHarmonicAdd	27	VelByPitch	19	ZoneOctaveShift	55
SubharmonicShift	27	VelClip	19	ZoneVelocityScale	22
SubharmonicTune	33	VelCompress	19	ZuckermanGate	52
SunnyNumberGate	50	VelCurve	19	ZumkellerGate	53
SuperabundantGate	50	VelDrift	19		